

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka
Specjalność: Inżynieria Systemów Informatycznych

PRACA DYPLOMOWA
MAGISTERSKA

**Projekt i analiza porównawcza wydajności skalowalnych architektur
systemów typu SaaS**

OSABOSA

Opiekun pracy
dr hab. inż. Imię Nazwisko

Słowa kluczowe: skalowalność, Kubernetes, HPA, mikroserwisy, SaaS, testy obciążeniowe,
Prometheus, autoskalowanie

Spis treści

1	Wstęp teoretyczny	1
1.1	Motywacja i kontekst badawczy	1
1.2	Problem badawczy i hipoteza	1
1.3	Cel i zakres pracy	1
1.4	Architektury skalowalne systemów	2
1.4.1	Monolit, SOA i mikroserwisy	2
1.4.2	Wzorce skalowania	2
1.5	Platformy orkiestracji kontenerów	3
1.5.1	Kubernetes — architektura i kluczowe komponenty	3
1.5.2	GKE — zarządzany Kubernetes w Google Cloud	3
1.6	Mechanizmy autoskalowania w Kubernetes	3
1.6.1	HorizontalPodAutoscaler (HPA)	3
1.6.2	VerticalPodAutoscaler (VPA)	4
1.6.3	KEDA	4
1.7	Modele SaaS i multi-tenancy	4
1.7.1	Software as a Service (SaaS)	4
1.7.2	Multi-tenancy	5
1.8	Przegląd literatury i istniejących badań	5
1.8.1	Badania nad HPA w Kubernetes	5
1.8.2	Luki badawcze	5
1.9	Struktura pracy	6
2	Przegląd technologii	7
2.1	Google Kubernetes Engine (GKE)	7
2.1.1	Charakterystyka	7
2.1.2	Uzasadnienie wyboru	7
2.2	Kubernetes jako platforma orkiestracji	8
2.2.1	Architektura	8
2.2.2	Kluczowe obiekty API	8
2.2.3	HorizontalPodAutoscaler (HPA) — szczegółowo	9
2.3	GCP LoadBalancer — ekspozycja serwisów	9
2.3.1	Mechanizm działania	9
2.3.2	Porównanie z ClusterIP i port-forward	9
2.4	Compute Engine — generator obciążenia	10
2.4.1	Rola w architekturze	10
2.4.2	Mapowanie hostów w k6	10
2.5	kube-prometheus-stack — stos monitorujący	10
2.5.1	Charakterystyka	10
2.5.2	Instalacja przez Helm	11

2.5.3	Zalety w porównaniu do ręcznych manifestów	11
2.5.4	Prometheus	12
2.5.5	Grafana	12
2.5.6	Prometheus Adapter	13
2.6	k6 — narzędzie do testów obciążeniowych	14
2.6.1	Charakterystyka	14
2.6.2	Scenariusze k6 w projekcie	14
2.7	Docker i konteneryzacja	15
2.7.1	Docker	15
2.7.2	Brak Docker Compose w środowisku badawczym	15
2.8	Podsumowanie stosu technologicznego	16
3	Projekt architektury systemu badawczego	17
3.1	Macierz badawcza	17
3.2	Aplikacje badawcze	18
3.2.1	CPU-service — obciążenie CPU-bound	18
3.2.2	IO-service — obciążenie I/O-bound	18
3.2.3	Echo-server — serwis pomocniczy	18
3.2.4	Memory-service — serwis pamięciowy (dodatkowy)	19
3.3	Strategie HPA	19
3.3.1	HPA CPU (Resource Metric)	19
3.3.2	HPA Memory (Resource Metric)	20
3.3.3	HPA Custom RPS (Pods Metric)	20
3.4	Architektura wdrożeniowa w GCP	21
3.4.1	Topologia sieciowa GCP	21
3.4.2	GKE Cluster — klaster Kubernetes	22
3.4.3	GCP LoadBalancer — ekspozycja serwisów	22
3.4.4	k6-generator — maszyna generująca obciążenie	23
3.4.5	Rejestr obrazów kontenerowych	23
3.5	Infrastruktura monitorująca	23
3.5.1	kube-prometheus-stack	23
3.5.2	Grafana — dashboard i eksport danych	24
3.5.3	Prometheus Adapter	24
3.6	Topologia zasobów Kubernetes	25
3.6.1	Zasoby przydzielone serwisom	25
3.7	Plan eksperymentów	25
3.7.1	Scenariusze testowe	25
3.7.2	Procedura eksperymentalna	26
3.7.3	Profile obciążeniowe k6	26
3.7.4	Metryki badawcze	27
3.7.5	Analiza zjawisk anomalnych	27
3.8	Podsumowanie architektury	28
4	Implementacja	29
4.1	Struktura repozytorium	29
4.2	Implementacja mikroservisów	30
4.2.1	Wspólne cechy implementacyjne	30
4.2.2	Budowanie i dystrybucja obrazów	30
4.2.3	CPU-service	31

4.2.4	IO-service	32
4.2.5	Echo-server	34
4.2.6	Memory-service	34
4.3	Konfiguracja Kubernetes	35
4.3.1	Namespace i struktura zasobów	35
4.3.2	Deployments	35
4.3.3	HPA	36
4.3.4	Services — LoadBalancer w GKE	37
4.4	Infrastruktura monitorująca	38
4.4.1	kube-prometheus-stack — instalacja przez Helm	38
4.4.2	Prometheus Adapter	38
4.4.3	Grafana — dashboard i eksport danych	40
4.5	Skrypty automatyzujące	40
4.5.1	run-tests.sh — orkiestracja testów	40
4.5.2	Uwaga o skryptach legacy	42
4.6	Scenariusze testowe k6	42
4.6.1	cpu-load-test.js	42
4.6.2	io-load-test.js	43
4.6.3	combined-scenario.js	43
4.7	Podsumowanie implementacji	44
5	Badania eksperymentalne i analiza wyników	47
5.1	Metodyka badań	47
5.1.1	Środowisko testowe — GKE	47
5.1.2	Procedura eksperymentalna	48
5.1.3	Zbierane metryki	48
5.1.4	Metryki badawcze — definicje operacyjne	49
5.2	Scenariusze testowe	49
5.2.1	Scenariusze bazowe (bez HPA)	49
5.2.2	Scenariusze CPU-bound	50
5.2.3	Scenariusze I/O-bound	50
5.3	Wyniki eksperymentalne — środowisko GKE	50
5.3.1	Scenariusze bazowe	50
5.3.2	Scenariusze CPU-bound — porównanie strategii HPA	51
5.3.3	Scenariusze I/O-bound — porównanie strategii HPA	51
5.3.4	Wykresy przebiegu skalowania	51
5.3.5	Analiza zjawisk anomalnych	51
5.4	Analiza wyników	53
5.4.1	Porównanie strategii HPA dla CPU-bound	53
5.4.2	Porównanie strategii HPA dla I/O-bound	53
5.4.3	Efektywność skalowania	54
5.4.4	Analiza zjawisk anomalnych — podsumowanie obserwacji	54
5.5	Dyskusja i interpretacja	54
5.6	Wnioski z badań	55
6	Rozszerzenie <i>SaaS Multi-Tenancy</i>	57
6.1	Status implementacji	57
6.2	Koncepcja <i>Multi-Tenancy</i> w środowisku GKE	57
6.2.1	Model izolacji tenantów	57

6.2.2	Projektowana architektura	58
6.3	Rozważania dotyczące wdrożenia na GKE	58
6.4	Planowane scenariusze testowe	59
6.5	Oczekiwane wyzwania	59
6.6	Podsumowanie	60
7	Podsumowanie	61
7.1	Podsumowanie zrealizowanych prac	61
7.2	Weryfikacja hipotezy badawczej	62
7.3	Ograniczenia pracy	62
7.4	Rekomendacje praktyczne	63
7.5	Dalsze kierunki badań	64
7.6	Podsumowanie końcowe	65
A	Spis listingów	67

Rozdział 1

Wstęp teoretyczny

1.1 Motywacja i kontekst badawczy

Współczesne systemy informatyczne funkcjonują w środowisku charakteryzującym się zmiennym obciążeniem, rosnącymi oczekiwaniami użytkowników dotyczącymi czasu odpowiedzi oraz koniecznością efektywnego wykorzystania zasobów sprzętowych. Model *SaaS* (*Software as a Service*) narzuca dodatkowe wymagania związane z obsługą wielu klientów (*tenantów*) współdzielących tę samą infrastrukturę przy zachowaniu izolacji danych i przewidywalnej wydajności.

Kluczowym mechanizmem umożliwiającym sprostanie tym wymaganiom jest automatyczne skalowanie (*autoscaling*) — zdolność systemu do dynamicznego dostosowywania liczby jednostek obliczeniowych do bieżącego obciążenia bez interwencji operatora. W ekosystemie *Kubernetes* podstawowym narzędziem realizującym skalowanie poziome jest *HorizontalPodAutoscaler* (HPA), który na podstawie obserwowanych metryk podejmuje decyzje o dodaniu lub usunięciu replik aplikacji.

1.2 Problem badawczy i hipoteza

Skuteczność mechanizmu HPA zależy od wyboru metryki, na podstawie której podejmowana jest decyzja o skalowaniu. Metryka musi być skorelowana z rzeczywistym obciążeniem aplikacji — w przeciwnym razie system może nie skalować się w ogóle (niedoskalowanie, *underscaling*) lub skalować się nadmiernie (przeskalowanie, *overscaling*). Problem ten jest szczególnie istotny w architekturach *SaaS*, gdzie różni tenantci generują obciążenia o odmiennych charakterystykach.

Hipoteza badawcza: Nie istnieje jedna uniwersalna metryka HPA — skuteczność strategii skalowania zależy od charakterystyki obciążeniowej aplikacji (*workload profile*). Błędny wybór metryki prowadzi do niedoskalowania lub przeskalowania.

1.3 Cel i zakres pracy

Celem niniejszej pracy jest:

1. Zaprojektowanie i implementacja prototypu skalowalnego systemu w modelu *SaaS*, składającego się z mikroservisów o diametralnie różnych profilach obciążeniowych (*CPU-bound* oraz *I/O-bound*).

2. Przeprowadzenie kontrolowanych eksperymentów porównujących trzy strategie HPA: opartą na zużyciu CPU, opartą na zużyciu pamięci oraz opartą na niestandardowej metryce RPS (*Requests Per Second*) dostarczanej przez *prometheus-adapter*.
3. Analiza porównawcza efektywności strategii skalowania w macierzy badawczej 2×3 (dwa typy obciążenia $\times$ trzy strategie HPA).
4. Sformułowanie rekomendacji dotyczących doboru metryki HPA w zależności od charakterystyki aplikacji.

1.4 Architektury skalowalne systemów

1.4.1 Monolit, SOA i mikroserwisy

Ewolucja architektur systemów rozproszonych przebiegała od monolitycznych aplikacji, przez architekturę zorientowaną na usługi (*SOA*), do współczesnego paradygmatu mikroserwisów. Każde z tych podejść oferuje inny kompromis między złożonością zarządzania a elastycznością skalowania:

- **Monolit** — cała funkcjonalność systemu zawarta w jednej jednostce wdrożeniowej. Skalowanie odbywa się przez uruchomienie wielu instancji całej aplikacji, co jest nieefektywne, gdy tylko wybrane komponenty wymagają większej mocy obliczeniowej.
- **SOA** (*Service-Oriented Architecture*) — system podzielony na usługi komunikujące się przez *Enterprise Service Bus* (ESB). Umożliwia selektywne skalowanie, ale wprowadza złożoność infrastrukturalną i potencjalne wąskie gardła w warstwie integracyjnej.
- **Mikroserwisy** — każda funkcja biznesowa jest niezależną jednostką wdrożeniową z własnym cyklem życia, stosem technologicznym i polityką skalowania. Model ten umożliwia precyzyjne skalowanie tylko tych komponentów, które są przeciążone, co maksymalizuje efektywność wykorzystania zasobów.

1.4.2 Wzorce skalowania

Skalowanie systemów informatycznych można realizować w trzech podstawowych wymiarach:

- **Skalowanie pionowe** (*vertical scaling*) — zwiększanie zasobów pojedynczej instancji (więcej CPU, RAM). Ograniczone fizycznymi możliwościami sprzętu; nie zwiększa odporności na awarie.
- **Skalowanie poziome** (*horizontal scaling*) — zwiększanie liczby instancji obsługujących ruch. Umożliwia rozłożenie obciążenia, zwiększa odporność na awarie, ale wymaga mechanizmów równoważenia obciążenia i koordynacji stanu.
- **Skalowanie automatyczne** (*autoscaling*) — dynamiczne dostosowywanie liczby instancji na podstawie obserwowanych metryk, bez interwencji operatora. Jest to kluczowy mechanizm w systemach chmurowych, gdzie koszty są proporcjonalne do wykorzystania zasobów.

1.5 Platformy orkiestracji kontenerów

1.5.1 Kubernetes — architektura i kluczowe komponenty

Kubernetes (K8s) jest otwartą platformą do automatyzacji wdrażania, skalowania i zarządzania aplikacjami kontenerowymi, pierwotnie opracowaną przez Google i obecnie utrzymaną przez *Cloud Native Computing Foundation* (CNCF).

Architektura *Kubernetes* opiera się na klastrze składającym się z:

- **Control Plane** — warstwa sterowania obejmująca *API server*, *etcd* (magazyn stanu), *scheduler* (przydzielanie Podów do węzłów) oraz *controller manager* (w tym *HorizontalPodAutoscaler*).
- **Worker Nodes** — węzły wykonawcze, na których uruchamiane są Pody za pośrednictwem *kubelet* i *container runtime* (np. *containerd*).

Podstawową jednostką obliczeniową w *Kubernetes* jest **Pod** — grupa jednego lub więcej kontenerów współdzielących przestrzeń sieciową i woluminy. Pody są efemeryczne — mogą być tworzone, niszczone i przenoszone między węzłami.

1.5.2 GKE — zarządzany Kubernetes w Google Cloud

W niniejszej pracy jako platformę badawczą wybrano *Google Kubernetes Engine* (GKE) — zarządzaną usługę *Kubernetes* oferowaną przez *Google Cloud Platform* (GCP). GKE automatyzuje zarządzanie warstwą sterowania (*Control Plane*), aktualizacje wersji, skalowanie węzłów oraz integrację z ekosystemem GCP (równoważenie obciążenia, sieci VPC, *Cloud Monitoring*). Wybór GKE podyktowany był następującymi czynnikami:

- **Pełna zgodność API** — GKE jest certyfikowaną dystrybucją *Kubernetes*, co oznacza, że wszystkie manifesty (*Deployment*, *HPA*, *Service*) działają identycznie jak na dowolnym innym klastrze zgodnym z CNCF.
- **Zarządzana warstwa sterowania** — GKE automatycznie zarządza *Control Plane*, eliminując konieczność ręcznej konfiguracji i utrzymania komponentów takich jak *etcd* czy *kube-apiserver*.
- **Integracja z GCP LoadBalancer** — serwisy typu *LoadBalancer* są automatycznie udostępniane przez globalną sieć GCP, co umożliwia bezpośrednią komunikację z zewnętrznymi klientami (w tym z maszyny generującej obciążenie testowe).
- **Elastyczność skalowania węzłów** — GKE umożliwia konfigurację *node pools* i automatyczne skalowanie węzłów (*cluster autoscaler*), co pozwala na dostosowanie infrastruktury do wymagań eksperymentów.

1.6 Mechanizmy autoskalowania w Kubernetes

1.6.1 HorizontalPodAutoscaler (HPA)

HorizontalPodAutoscaler jest kontrolerem *Kubernetes*, który automatycznie dostosowuje liczbę replik w *Deployment*, *ReplicaSet* lub *StatefulSet* na podstawie obserwowanych metryk. Algorytm HPA działa według formuły:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil \quad (1.1)$$

HPA w wersji API `autoscaling/v2` obsługuje trzy typy metryk:

1. **Resource metrics** — metryki zasobów kontenera (CPU, pamięć) dostarczane przez *metrics-server*.
2. **Pods metrics** — metryki niestandardowe skojarzone z konkretnymi Podami, dostarczane przez zewnętrzne adaptory (np. *prometheus-adapter*).
3. **Object metrics** — metryki opisujące obiekty *Kubernetes* (np. *Ingress*), również dostarczane przez adaptory.

1.6.2 VerticalPodAutoscaler (VPA)

VerticalPodAutoscaler (VPA) automatycznie dostosowuje *resource requests* i *limits* dla kontenerów w Podzie. W przeciwieństwie do HPA, który zmienia liczbę replik, VPA zmienia ilość zasobów przydzielonych każdej replice. VPA nie jest przedmiotem badań w niniejszej pracy, ale stanowi uzupełniający mechanizm skalowania.

1.6.3 KEDA

KEDA (*Kubernetes Event-Driven Autoscaling*) jest komponentem umożliwiającym skalowanie na podstawie zdarzeń z zewnętrznych systemów (np. długość kolejki *Apache Kafka*, liczba wiadomości w *RabbitMQ*). *KEDA* rozszerza możliwości HPA o skalowanie do zera replik oraz integrację z bogatym ekosystemem *scalerów*. *KEDA* nie jest przedmiotem badań w niniejszej pracy, ale stanowi istotny kierunek dalszych prac.

1.7 Modele SaaS i multi-tenancy

1.7.1 Software as a Service (SaaS)

Model *SaaS* jest jedną z trzech podstawowych kategorii *cloud computing* (obok *IaaS* i *PaaS*), w której dostawca udostępnia użytkownikom gotową aplikację przez Internet, zarządzając całą infrastrukturą, środowiskiem uruchomieniowym i danymi. Kluczowe cechy modelu *SaaS* to:

- **Współdzielenie infrastruktury** — wielu klientów (*tenantów*) korzysta z tej samej instancji aplikacji i infrastruktury.
- **Izolacja danych** — dane każdego tenanta muszą być logicznie odseparowane.
- **Skalowalność** — system musi obsługiwać zmienne obciążenie od wielu tenantów jednocześnie.
- **Rozliczanie** — model subskrypcyjny (miesięczny/roczny) lub *pay-as-you-go*.

1.7.2 Multi-tenancy

Multi-tenancy (wielodostępność) jest architektonicznym wzorcem umożliwiającym obsługę wielu klientów przez pojedynczą instancję systemu. W kontekście *Kubernetes* można wyróżnić kilka poziomów izolacji:

- **Namespace per tenant** — każdy tenant otrzymuje własny *namespace* z izolowanymi zasobami (*ResourceQuota*, *NetworkPolicy*).
- **Application-level multi-tenancy** — tenanci są rozróżniani na poziomie aplikacji (np. przez nagłówek *X-Tenant-ID*), współdzieląc ten sam *namespace*.
- **Cluster per tenant** — każdy tenant otrzymuje dedykowany klaster, co zapewnia najwyższy poziom izolacji przy najwyższym koszcie.

1.8 Przegląd literatury i istniejących badań

Problematyka autoskalowania w *Kubernetes* była przedmiotem licznych badań w ostatnich latach. Poniżej przedstawiono najważniejsze kierunki badawcze i luki, które motywują niniejszą pracę.

1.8.1 Badania nad HPA w Kubernetes

Casalicchio i Perciballi [1] przeprowadzili analizę porównawczą domyślnego HPA *Kubernetes* z *Kubernetes Event-Driven Autoscaling* (KEDA), badając czas reakcji i wykorzystanie zasobów dla obciążeń o różnych profilach. Autorzy wykazali, że KEDA oferuje szybszy czas skalowania przy obciążeniach zdarzeniowych, ale domyślny HPA jest wystarczający dla obciążeń o stałym charakterze.

Nguyen i Kim [2] zaproponowali predykcyjny algorytm skalowania oparty na sieciach LSTM (*Long Short-Term Memory*), który przewiduje przyszłe obciążenie i skaluje proaktywnie, redukując liczbę naruszeń SLA o 30% w porównaniu do reaktywnego HPA. Podejście to wymaga jednak znacznych zasobów obliczeniowych na trenowanie modelu.

1.8.2 Luki badawcze

Na podstawie przeglądu literatury zidentyfikowano następujące luki badawcze:

1. **Brak systematycznego porównania strategii HPA dla różnych profili obciążeń** — większość badań testuje HPA na jednorodnym obciążeniu (np. wyłącznie CPU-bound), nie analizując interakcji między profilem obciążeniowym a wyborem metryki.
2. **Ograniczona analiza HPA Memory** — strategia oparta na zużyciu pamięci jest rzadko badana, ponieważ pamięć w większości aplikacji nie koreluje z obciążeniem — niniejsza praca dostarcza empirycznych dowodów na tę tezę.
3. **Brak badań łączących HPA z modelem SaaS** — istniejące badania nad *multi-tenancy* koncentrują się na izolacji danych i bezpieczeństwie, pomijając wpływ wielodostępności na efektywność autoskalowania.

4. **Brak analizy zjawisk anomalnych w autoskalowaniu chmurowym** — istniejące badania koncentrują się na metrykach wydajnościowych (RPS, latency), pomijając analizę zjawisk przejściowych takich jak opóźnienie reakcji HPA (*HPA Lag*), opóźnienie zimnego startu Podów (*Cold Start Latency*) czy kaskadowe błędy przy *CPU throttlingu*. Niniejsza praca identyfikuje i dokumentuje te zjawiska na podstawie eksperymentów w środowisku GKE.
5. **Brak otwartego, odtwarzalnego środowiska badawczego** — większość eksperymentów jest trudna do odtworzenia ze względu na zamknięty kod lub specyficzne dla dostawcy chmury konfiguracje. Niniejsza praca udostępnia kompletne repozytorium kodu, konfiguracji *Kubernetes* i skryptów testowych, umożliwiając odtworzenie środowiska na dowolnym klastrze GKE.

1.9 Struktura pracy

Praca składa się z siedmiu rozdziałów:

- **Rozdział 1** (niniejszy) — wprowadzenie teoretyczne, motywacja, hipoteza badawcza, przegląd literatury.
- **Rozdział 2** — szczegółowy przegląd technologii wykorzystanych w projekcie: *Kubernetes*, *GKE*, *Prometheus*, *Grafana*, *k6*, *Docker*, *kube-prometheus-stack*, *GCP LoadBalancer*, *Compute Engine*.
- **Rozdział 3** — projekt architektury systemu badawczego: macierz badawcza, specyfikacja aplikacji, strategie HPA, infrastruktura monitorująca, plan eksperymentów.
- **Rozdział 4** — implementacja: struktura kodu, implementacja serwisów, konfiguracja *Kubernetes*, monitoring, skrypty automatyzujące, scenariusze testowe.
- **Rozdział 5** — badania eksperymentalne i analiza wyników: metodyka, scenariusze testowe, wyniki, interpretacja, porównanie strategii.
- **Rozdział 6** — rozszerzenie *SaaS Multi-Tenancy*: koncepcja, implementacja izolacji tenantów, testy wielodostępne, wyniki i analiza.
- **Rozdział 7** — podsumowanie, wnioski, rekomendacje, ograniczenia badań i dalsze kierunki prac.

Rozdział 2

Przegląd technologii

Rozdział niniejszy przedstawia technologie wykorzystane w projekcie badawczym, ich architekturę, kluczowe cechy oraz uzasadnienie wyboru każdego komponentu. Całość infrastruktury badawczej działa w środowisku *Google Cloud Platform* (GCP), z klastrem *Google Kubernetes Engine* (GKE) jako platformą orkiestracji oraz dedykowaną maszyną *Compute Engine* jako generatorem obciążenia testowego.

2.1 Google Kubernetes Engine (GKE)

2.1.1 Charakterystyka

Google Kubernetes Engine (GKE) jest zarządzaną usługą *Kubernetes* oferowaną przez *Google Cloud Platform* (GCP). GKE automatyzuje zarządzanie warstwą sterowania (*Control Plane*), aktualizacje wersji, skalowanie węzłów oraz integrację z ekosystemem GCP.

Architektura GKE obejmuje:

- **Control Plane** — zarządzana przez GCP warstwa sterowania, niewidoczna dla użytkownika. GKE automatycznie zarządza `kube-apiserver`, `etcd`, `kube-scheduler` i `kube-controller-manager`. Użytkownik nie ma bezpośredniego dostępu do tych komponentów.
- **Worker Nodes** — węzły wykonawcze uruchamiane na maszynach wirtualnych *Compute Engine*. W projekcie wykorzystano maszyny typu `e2-standard-4` (4 vCPU, 16 GB RAM) w konfiguracji VPC-native.
- **VPC-native cluster** — klastrowe GKE z natywną integracją z *Virtual Private Cloud* (VPC) GCP, umożliwiającą bezpośrednią komunikację między zasobami w tej samej sieci VPC.

2.1.2 Uzasadnienie wyboru

GKE został wybrany jako jedyna platforma badawcza z następujących powodów:

- **Pełna zgodność API** — GKE jest certyfikowaną dystrybucją *Kubernetes* przez CNCF, co gwarantuje, że wszystkie manifesty (Deployment, HPA, Service) działają zgodnie ze standardem.
- **Zarządzana warstwa sterowania** — eliminacja konieczności ręcznej instalacji i utrzymania *Control Plane*, co redukuje ryzyko błędów konfiguracyjnych.

- **Integracja z GCP LoadBalancer** — serwisy typu LoadBalancer są automatycznie udostępniane przez globalną infrastrukturę GCP, co umożliwia bezpośrednią komunikację z maszyną generującą obciążenie testowe.
- **Współdzielona sieć VPC** — maszyna *Compute Engine* z *k6* znajduje się w tej samej sieci VPC co klastery GKE, co zapewnia niskie opóźnienie sieciowe ($\sim 1\text{--}5$ ms) między generatorem obciążenia a serwisami.
- **Elastyczność skalowania węzłów** — GKE *cluster autoscaler* automatycznie dostosowuje liczbę węzłów do zapotrzebowania, umożliwiając testy z dużą liczbą Podów.

2.2 Kubernetes jako platforma orkiestracji

2.2.1 Architektura

Kubernetes (K8s) jest otwartoźródłową platformą do orkiestracji kontenerów, zarządzającą cyklem życia aplikacji w klastrze maszyn. Architektura *Kubernetes* dzieli się na dwie główne warstwy:

Control Plane — warstwa sterowania odpowiedzialna za utrzymanie stanu klastra:

- **kube-apiserver** — centralny punkt komunikacji, wystawia API *Kubernetes* (REST).
- **etcd** — rozproszony magazyn klucz-wartość przechowujący cały stan klastra.
- **kube-scheduler** — przypisuje nowe Pody do węzłów na podstawie dostępnych zasobów i reguł.
- **kube-controller-manager** — uruchamia kontrolery, w tym *HorizontalPodAutoscaler*, *ReplicaSet controller* i inne.

Worker Nodes — węzły wykonawcze:

- **kubelet** — agent zarządzający Podami na węźle, komunikuje się z *API server*.
- **kube-proxy** — zarządza regułami sieciowymi, implementuje *Service* abstrakcję.
- **container runtime** — uruchamia kontenery (np. *containerd*).

2.2.2 Kluczowe obiekty API

- **Pod** — najmniejsza jednostka obliczeniowa; grupa kontenerów ze współdzieloną przestrzenią sieciową i woluminami.
- **Deployment** — deklaratywne zarządzanie replikami Podów z *rolling updates* i *rollback*.
- **Service** — stabilny punkt dostępowy (ClusterIP, NodePort, LoadBalancer) do zestawu Podów.
- **ConfigMap** i **Secret** — zewnętrzna konfiguracja aplikacji.
- **HorizontalPodAutoscaler** — automatyczne skalowanie liczby replik na podstawie metryk.
- **Namespace** — logiczna izolacja zasobów w klastrze.

2.2.3 HorizontalPodAutoscaler (HPA) — szczegółowo

HPA okresowo (domyślnie co 15 sekund) pobiera wartości metryk dla Podów w *Deployment* i oblicza pożądaną liczbę replik. Algorytm skalowania:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil \quad (2.1)$$

Dodatkowo, HPA implementuje mechanizmy stabilizacyjne:

- **Scale-down stabilization window** — domyślnie 5 minut (300 s); HPA czeka tyle czasu przed podjęciem decyzji o skalowaniu w dół, uwzględniając maksymalną wartość replik z tego okna.
- **Tolerance** — domyślnie 10%; metryka musi odchylić się o więcej niż 10% od progu, aby wywołać skalowanie.
- **Cooldown/scaling policies** — od wersji *autoscaling/v2* można definiować polityki ograniczające tempo skalowania (np. maksymalnie 2 repliki na 60 sekund).

2.3 GCP LoadBalancer — ekspozycja serwisów

2.3.1 Mechanizm działania

W środowisku GKE, serwisy typu *LoadBalancer* są automatycznie integrowane z infrastrukturą równoważenia obciążenia GCP. GKE tworzy zewnętrzny *Google Cloud LoadBalancer* (Layer 4, TCP), który otrzymuje publiczny adres IP i przekazuje ruch do Podów w klastrze.

Proces tworzenia *LoadBalancer* w GKE:

1. Użytkownik tworzy *Service* typu *LoadBalancer*.
2. GKE *cloud controller manager* wykrywa nowy serwis i tworzy regułę *forwarding rule* w GCP.
3. GCP przydziela publiczny adres IP (regionalny, statyczny lub efemeryczny).
4. Ruch przychodzący na ten adres IP jest dystrybuowany do Podów przez *kube-proxy* na węzłach.

W projekcie wszystkie serwisy aplikacyjne (*cpu-service*, *io-service*, *echo-server*) są eksponowane przez *LoadBalancer*, co umożliwia maszynie *k6-generator* bezpośrednie wysyłanie żądań HTTP bez konieczności tunelowania (*kubectl port-forward*) czy konfiguracji DNS.

2.3.2 Porównanie z ClusterIP i port-forward

W środowiskach deweloperskich często stosuje się *kubectl port-forward* do przekierowania ruchu z lokalnego *localhost* na serwis w klastrze. Rozwiązanie to jest wygodne, ale nieodpowiednie dla testów obciążeniowych z kilku powodów:

- **Ograniczona przepustowość** — *port-forward* używa pojedynczego połączenia *Websocket* przez *API server*, co tworzy wąskie gardło przy dużym obciążeniu.

- **Brak wysokiej dostępności** — awaria procesu `port-forward` przerywa test.
- **Nierealistyczne opóźnienia** — tunelowanie przez *API server* dodaje dodatkowe opóźnienie, które nie występuje w środowisku produkcyjnym.

GCP LoadBalancer eliminuje te ograniczenia, zapewniając bezpośrednią ścieżkę sieciową między generatorem obciążenia a Podami, co odzwierciedla rzeczywiste warunki produkcyjne.

2.4 Compute Engine — generator obciążenia

2.4.1 Rola w architekturze

Google Compute Engine (GCE) jest usługą maszyn wirtualnych GCP. W projekcie wykorzystano dedykowaną maszynę *Compute Engine* (nazwaną **k6-generator**) jako hosta dla narzędzia *k6*, generującego obciążenie testowe.

Maszyna **k6-generator** znajduje się w tej samej sieci VPC co klaster GKE, co zapewnia:

- **Niskie opóźnienie sieciowe** — ruch między maszynami w tej samej sieci VPC nie opuszcza infrastruktury GCP, co minimalizuje opóźnienia.
- **Bezpośrednią komunikację** — *k6* wysyła żądania bezpośrednio na adresy IP *GCP LoadBalancer*, mapowane w konfiguracji `options.hosts` skryptów testowych.
- **Izolację obciążenia** — generator obciążenia działa na osobnej maszynie, nie konkurując o zasoby z klastrem *Kubernetes*, co zapobiega zakłócaniu wyników testów.

2.4.2 Mapowanie hostów w k6

Zamiast konfiguracji DNS (która wymagałaby rejestracji domen i propagacji rekordów), *k6* używa mechanizmu `options.hosts` do bezpośredniego mapowania nazw serwisów na adresy IP *GCP LoadBalancer*:

```

1 export const options = {
2   hosts: {
3     'cpu-service': '34.118.71.122',    // GCP LoadBalancer IP
4     'io-service': '35.204.93.45',    // GCP LoadBalancer IP
5   },
6   scenarios: { ... }
7 };

```

Listing 2.1 Mapowanie hostów w `options.hosts` skryptu *k6*

Mechanizm ten działa na poziomie biblioteki HTTP *k6* (napisanej w *Go*) i podmienia nagłówki `Host` oraz docelowy adres IP dla każdego żądania, eliminując potrzebę zewnętrznego DNS.

2.5 kube-prometheus-stack — stos monitorujący

2.5.1 Charakterystyka

kube-prometheus-stack jest pakietem *Helm* (*Helm chart*) rozwijanym przez społeczność *prometheus-community*, który instaluje kompletny stos monitorujący dla *Kubernetes*:

- **Prometheus** — system zbierania i przechowywania metryk w postaci szeregów czasowych.
- **Grafana** — platforma wizualizacji danych z predefiniowanymi *dashboardsami*.
- **Alertmanager** — system alertowania (nieaktywny w środowisku badawczym).
- **kube-state-metrics** — komponent generujący metryki o stanie obiektów *Kubernetes* (Deploymenty, HPA, Pody, Węzły).
- **node-exporter** — komponent zbierający metryki systemowe z węzłów (CPU, pamięć, dysk, sieć).
- **Prometheus Operator** — kontroler *Kubernetes* zarządzający cyklem życia instancji *Prometheusa* i reguł (*PrometheusRule*, *ServiceMonitor*, *PodMonitor*).

2.5.2 Instalacja przez Helm

Pakiet jest instalowany jednym poleceniem *Helm*:

```
1 helm repo add prometheus-community \  
2   https://prometheus-community.github.io/helm-charts  
3 helm repo update  
4 helm install monitoring prometheus-community/kube-prometheus-stack \  
5   --namespace magisterka --create-namespace \  
6   --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=  
7     false \  
   --set grafana.service.type=LoadBalancer
```

Listing 2.2 Instalacja *kube-prometheus-stack* przez Helm

Parametr `grafana.service.type=LoadBalancer` eksponuje *Grafanę* publicznie przez GCP LoadBalancer, umożliwiając obserwację eksperymentów w czasie rzeczywistym z dowolnej przeglądarki.

2.5.3 Zalety w porównaniu do ręcznych manifestów

W odróżnieniu od ręcznego wdrażania *Prometheusa* i *Grafany* przez pojedyncze manifesty YAML, *kube-prometheus-stack* oferuje:

- **Automatyczną konfigurację** — *ServiceMonitor* i *PodMonitor* automatycznie wykrywają źródła metryk na podstawie etykiet, bez konieczności ręcznej edycji `prometheus.yml`.
- **Predefiniowane dashboardy** — *Grafana* zawiera zestaw gotowych dashboardów dla *Kubernetes* (m.in. *Kubernetes / Compute Resources / Pod*, *Kubernetes / Networking / Cluster*).
- **Zarządzanie cyklem życia** — *Prometheus Operator* zarządza aktualizacjami, backupami i skalowaniem *Prometheusa*.
- **Spójność wersji** — wszystkie komponenty są testowane razem, co eliminuje problemy z niezgodnością wersji.

2.5.4 Prometheus

Prometheus jest otwartoźródłowym systemem monitorowania i alertowania, pierwotnie opracowanym w *SoundCloud*, obecnie inkubowanym przez CNCF. Architektura *Prometheusa* opiera się na:

- **Modelu pull** — *Prometheus* aktywnie pobiera (*scrape*) metryki z aplikacji poprzez HTTP w interwałach (w niniejszym projekcie: 15 s).
- **TSDB** (*Time Series Database*) — natywna baza szeregów czasowych przechowująca dane z etykietami (*labels*).
- **PromQL** — funkcyjny język zapytań umożliwiający agregacje, operacje na zakresach czasowych, predykcję.
- **Service Discovery** — automatyczne wykrywanie celów (*targets*) do scrapowania przez integrację z API *Kubernetes* oraz przez *ServiceMonitor*.

W projekcie *Prometheus* pełni podwójną rolę:

1. Zbiera metryki infrastrukturalne z *Kubernetes* (CPU, RAM, liczba replik, stan HPA) poprzez `kube-state-metrics` i `node-exporter`.
2. Zbiera metryki aplikacyjne z `cpu-service` i `io-service` (RPS, czas przetwarzania, liczba aktywnych zapytań) z endpointów `/metrics`.
3. Udostępnia te metryki przez API dla *prometheus-adapter*, który przekształca je na format zrozumiały dla HPA *Kubernetes*.

2.5.5 Grafana

Grafana jest platformą do wizualizacji danych i tworzenia *dashboardów*, integrującą się z wieloma źródłami danych (m.in. *Prometheus*, *InfluxDB*, *Elasticsearch*). W projekcie wykorzystano:

- **Grafana eksponowana przez GCP LoadBalancer** — typ serwisu *LoadBalancer* umożliwia dostęp do dashboardów z dowolnej przeglądarki bez tunelowania.
- **Dashboard „Magisterka — System Overview”** — autorski dashboard zawierający panele pokazujące kluczowe metryki: RPS, zużycie CPU per Pod, latency p50/p95/p99, liczbę Podów, stan HPA.
- **Eksport CSV** — dla każdego scenariusza testowego eksportowanych jest 5 zestawów danych: liczba replik HPA, CPU per Pod, pamięć per Pod, RPS (przepustowość) oraz opóźnienia (latency) w czasie. Dane te służą jako podstawa analizy w Rozdziale 5.

2.5.6 Prometheus Adapter

Prometheus Adapter (registry.k8s.io/prometheus-adapter/prometheus-adapter:v0.12.0) jest komponentem implementującym *Kubernetes Custom Metrics API* i *External Metrics API*. Jego rola polega na:

1. Rejestracji jako *APIService* w *Kubernetes*, rozszerzając API o ścieżki `/apis/custom.metrics.k8s`
2. Pobieraniu metryk z *Prometheusa* poprzez zapytania *PromQL*.
3. Serwowaniu tych metryk w formacie oczekiwanym przez HPA.

Konfiguracja adaptera jest przechowywana w `ConfigMap` (`prometheus-adapter-config`) i zawiera reguły mapowania metryk *Prometheusa* na metryki *Kubernetes*:

```

1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: prometheus-adapter-config
5    namespace: magisterka
6    labels:
7      app: prometheus-adapter
8      tier: monitoring
9  data:
10   config.yaml: |
11     rules:
12       - seriesQuery: 'cpu_service_http_requests_total{namespace!="",pod
13         !=""}'
14       resources:
15         overrides:
16           namespace: {resource: "namespace"}
17           pod: {resource: "pod"}
18       name:
19         matches: "cpu_service_http_requests_total"
20         as: "http_requests_per_second"
21       metricsQuery: 'sum(rate(cpu_service_http_requests_total{<<.
22         LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
23     - seriesQuery: 'cpu_service_http_requests_inprogress{namespace!="",
24       pod!=""}'
25     resources:
26       overrides:
27         namespace: {resource: "namespace"}
28         pod: {resource: "pod"}
29     name:
30       matches: "cpu_service_http_requests_inprogress"
31       as: "http_requests_inflight"
32     metricsQuery: 'sum(cpu_service_http_requests_inprogress{<<.
33       LabelMatchers>>}) by (<<.GroupBy>>)'
34     - seriesQuery: 'io_service_http_requests_total{namespace!="",pod!=""}
35       ,
36     resources:
37       overrides:
38         namespace: {resource: "namespace"}
39         pod: {resource: "pod"}
40     name:
41       matches: "io_service_http_requests_total"
42       as: "http_requests_per_second"
43     metricsQuery: 'sum(rate(io_service_http_requests_total{<<.
44       LabelMatchers>>}[1m])) by (<<.GroupBy>>)'

```

```

39   - seriesQuery: 'io_service_http_requests_inprogress{namespace!="",pod
    !=""}'
40   resources:
41     overrides:
42       namespace: {resource: "namespace"}
43       pod: {resource: "pod"}
44   name:
45     matches: "io_service_http_requests_inprogress"
46     as: "http_requests_inflight"
47   metricsQuery: 'sum(io_service_http_requests_inprogress{<<.
    LabelMatchers>>}) by (<<.GroupBy>>)'
48 - seriesQuery: '{__name__=~"^container_.*",namespace!="",pod!=""}'
49   resources:
50     overrides:
51       namespace: {resource: "namespace"}
52       pod: {resource: "pod"}
53   name:
54     matches: "^container_(.*)$"
55     as: ""
56   metricsQuery: 'sum(<<.Series>>{<<.LabelMatchers>>,container!="POD",
    container!=""}) by (<<.GroupBy>>)'

```

Listing 2.3 Konfiguracja *Prometheus Adapter* — reguły mapowania (plik `k8s/monitoring/prometheus-adapter-config.yaml`)

2.6 k6 — narzędzie do testów obciążeniowych

2.6.1 Charakterystyka

k6 (*Grafana k6*) jest otwartoźródłowym narzędziem do testów obciążeniowych, napisanym w *Go* z interfejsem skryptowym w *JavaScript*. Kluczowe cechy *k6*:

- **Skryptowalność** — testy definiowane w *JavaScript* (ES6), z dostępem do modułów `http`, `check`, `sleep`, `encoding`.
- **Executory** — różne modele obciążenia: `ramping-vus` (stopniowa zmiana liczby VU), `constant-vus` (stała liczba VU).
- **Thresholds** — progi akceptacji (np. `p(95) < 5000`) których przekroczenie oznacza niepowodzenie testu.
- **Wydaźność** — pojedyncza instancja *k6* generuje setki do tysięcy VU (*Virtual Users*) bez potrzeby rozproszonej architektury.
- **Wyniki** — eksport do JSON, CSV, tekstowe podsumowanie na zakończenie testu.
- **Mapowanie hostów** — mechanizm `options.hosts` umożliwia bezpośrednie mapowanie nazw na adresy IP bez konfiguracji DNS.

2.6.2 Scenariusze k6 w projekcie

W projekcie zdefiniowano trzy skrypty *k6*:

- `cpu-load-test.js` — dwa scenariusze: Fibonacci (ramp-up 1→50 VU, steady 120 s) oraz przetwarzanie obrazu (stałe 10 VU).

- `io-load-test.js` — trzy scenariusze: zapytania ze stałym opóźnieniem (ramp-up 1→100 VU), test obciążeniowy z rosnącym opóźnieniem (50→2000 ms), wywołania zewnętrzne (`external_call=true`).
- `combined-scenario.js` — trzy równoległe scenariusze: Fibonacci (30 VU), przetwarzanie obrazu (5 VU), zapytania IO (80 VU).

2.7 Docker i konteneryzacja

2.7.1 Docker

Docker jest platformą do tworzenia, dystrybucji i uruchamiania kontenerów. W projekcie każdy mikroservis jest konteneryzowany za pomocą *Dockerfile*:

- Obraz bazowy: `python:3.12-slim` (minimalny rozmiar, ~150 MB).
- Instalacja zależności przez `pip install -r requirements.txt`.
- Uruchomienie przez `uvicorn` (serwer ASGI) z parametrami: `--host 0.0.0.0 --port 8000`.

Obrazy Docker są budowane lokalnie i przesyłane do rejestru kontenerów GCP (*Artifact Registry* lub *Container Registry*), skąd są pobierane przez GKE podczas wdrażania. Alternatywnie, obrazy mogą być budowane bezpośrednio w GCP za pomocą *Cloud Build*.

2.7.2 Brak Docker Compose w środowisku badawczym

W odróżnieniu od typowego procesu deweloperskiego, w niniejszym projekcie nie wykorzystuje się *Docker Compose* do lokalnej orkiestracji. Wszystkie testy są przeprowadzane bezpośrednio na klastrze GKE, co zapewnia zgodność środowiska testowego z produkcyjnym. Plik `docker-compose.yaml` znajdujący się w repozytorium służy wyłącznie do opcjonalnego, szybkiego uruchomienia serwisów w środowisku deweloperskim i nie jest wykorzystywany w badaniach.

2.8 Podsumowanie stosu technologicznego

Warstwa	Technologia	Opis
Język / Framework	Python 3.12 + FastAPI	Implementacja mikroservisów
Serwer ASGI	Uvicorn	Serwer HTTP dla FastAPI
Konteneryzacja	Docker	Budowa obrazów
Rejestr obrazów	GCP Artifact Registry	Przechowywanie obrazów
Orkiestracja	GKE (Google Kubernetes Engine)	Zarządzany klaster <i>Kubernetes</i> w GCP
Sieć	GCP VPC + LoadBalancer	Łączność między komponentami, ekspozycja serwisów
Monitoring	kubernetes-stack (Helm)	Prometheus + Grafana + kube-state-metrics
Custom Metrics	prometheus-adapter v0.12.0	Mostek Prometheus → HPA
Testy obciążeniowe	k6 v0.48+ (na Compute Engine)	Generowanie ruchu, pomiar metryk
Generator obciążenia	GCP Compute Engine	Dedykowana VM dla <i>k6</i>

Tabela 2.1 Podsumowanie stosu technologicznego projektu — środowisko GCP/GKE

Rozdział 3

Projekt architektury systemu badawczego

Rozdział niniejszy opisuje projekt architektury systemu badawczego — jego komponenty, strategię skalowania, infrastrukturę monitorującą oraz plan eksperymentów. Wszystkie decyzje projektowe zostały podporządkowane celowi badawczemu: porównaniu skuteczności trzech strategii *HorizontalPodAutoscaler* (HPA) dla dwóch diametralnie różnych profili obciążeniowych.

3.1 Macierz badawcza

Podstawą metodologii badawczej jest macierz 2×3 obejmująca dwie aplikacje o odmiennych profilach obciążeniowych i trzy strategie HPA:

Profil aplikacji	HPA CPU (50%)	HPA Memory (70%)	HPA Custom RPS (100/Pod)
<i>CPU-bound</i> (cpu-service)	cpu-cpu	cpu-memory	cpu-custom
<i>I/O-bound</i> (io-service)	io-cpu	io-memory	io-custom

Tabela 3.1 Macierz badawcza 2×3 — identyfikatory scenariuszy testowych

Każda z sześciu kombinacji jest testowana w odrębnym scenariuszu, co daje łącznie sześć scenariuszy właściwych (plus dwa scenariusze bazowe bez HPA). Macierz umożliwia izolowanie wpływu pojedynczej zmiennej niezależnej (strategia HPA) przy kontrolowanym profilu obciążeniowym.

Oprócz sześciu scenariuszy macierzy badawczej zdefiniowano dwa scenariusze bazowe (*baseline*):

- **baseline-cpu** — *CPU-bound*, brak HPA, 1 replika na stałe.
- **baseline-io** — *I/O-bound*, brak HPA, 1 replika na stałe.

Scenariusze bazowe służą jako punkt odniesienia do oceny, czy HPA w ogóle poprawia wydajność w stosunku do statycznej konfiguracji. Dodatkowo, dla testów integracyjnych zdefiniowano scenariusz łączony (**combined**), w którym oba serwisy są obciążane jednocześnie.

3.2 Aplikacje badawcze

System składa się z trzech mikroserwisów — dwóch podlegających badaniu (`cpu-service`, `io-service`) oraz jednego pomocniczego (`echo-server`). Każdy mikroserwis jest niezależną aplikacją *FastAPI* (Python 3.12) konteneryzowaną za pomocą *Docker* i wdrażaną jako *Deployment* w *Kubernetes*.

3.2.1 CPU-service — obciążenie CPU-bound

`cpu-service` implementuje dwa endpointy obciążające procesor:

- `POST /process` — przetwarzanie obrazu: akceptuje plik graficzny (*multipart form*), wykonuje sekwencję operacji CPU-intensywnych z użyciem biblioteki *Pillow* (m.in. konwersja do skali szarości, filtr `BLUR`, filtr `SHARPEN`, filtr `FIND_EDGES`, ekstrakcja kanałów RGB, `resize`), oblicza entropię Shannona dla każdego kanału, zwraca statystyki przetwarzania (wymiary, entropia, czas).
- `GET /fibonacci` — obliczanie n -tej liczby Fibonacciego rekurencyjnym algorytmem o złożoności $O(2^n)$. Parametr n jest konfigurowalny (domyślnie $n = 25$).

Obie operacje są czysto CPU-intensywne — nie wykonują operacji sieciowych ani dyskowych poza odbiorem żądania i wysłaniem odpowiedzi. Dzięki temu zużycie CPU silnie koreluje z obciążeniem żądaniami.

3.2.2 IO-service — obciążenie I/O-bound

`io-service` implementuje dwa endpointy symulujące operacje wejścia-wyjścia:

- `GET /query` — łańcuch asynchronicznych opóźnień: wykonuje sekwencję wywołań `asyncio.sleep()` z konfigurowalnym opóźnieniem bazowym (`delay`, domyślnie 100 ms), liczbą kroków (`steps`, domyślnie 3) oraz losowym rozrzutem (*jitter*, domyślnie $\pm 20\%$). Całkowity czas odpowiedzi jest sumą opóźnień, a procesor pozostaje bezczynny podczas oczekiwania.
- `GET /upstream` — wywołanie zewnętrzne: przekazuje żądanie do `echo-server` i zwraca odpowiedź z dodatkowym opóźnieniem symulującym przetwarzanie.

Serwis wykorzystuje `asyncio.Semaphore(50)` do ograniczenia liczby współbieżnych zapytań, co zapobiega wyczerpaniu zasobów systemowych przy bardzo wysokim obciążeniu.

3.2.3 Echo-server — serwis pomocniczy

`echo-server` jest minimalistycznym serwisem pełniącym rolę zależności zewnętrznej dla `io-service`. Udostępnia dwa endpointy:

- `GET /health` — *health check*.
- `POST /echo` — zwraca kopię otrzymanego żądania (nagłówki, body, parametry), umożliwiając weryfikację poprawnego przekazywania danych.

Echo-server jest wyłączony z macierzy badawczej — służy wyłącznie jako cel wywołań w scenariuszu `external_calls` dla `io-service`.

3.2.4 Memory-service — serwis pamięciowy (dodatkowy)

W ramach rozwoju projektu zaimplementowano również `memory-service` — serwis z pamięciową bazą klucz-wartość (*in-memory cache*) symulujący obciążenie typu *memory-bound*. Udostępnia endpointy `GET/POST/DELETE /cache`, `POST /cache/fill` (wypełnienie cache danymi testowymi o kontrolowanym rozmiarze) oraz `GET /stats` (statystyki trafień). `memory-service` nie został włączony do głównej macierzy badawczej na obecnym etapie prac — stanowi rozszerzenie do potencjalnych przyszłych badań.

3.3 Strategie HPA

Dla każdej aplikacji badawczej zdefiniowano trzy warianty *HorizontalPodAutoscaler*, różniące się wyłącznie typem metryki. Wszystkie HPA współdzielą następujące parametry:

- `minReplicas`: 2
- `maxReplicas`: 10
- `apiVersion`: `autoscaling/v2`

3.3.1 HPA CPU (Resource Metric)

Strategia oparta na zużyciu CPU — HPA monitoruje średnie zużycie procesora jako procent *resource request* dla wszystkich Podów w *Deployment*. Wartość progowa: 50%. Oznacza to, że jeśli średnie zużycie CPU przekracza 50% `requests` (czyli 125 m dla `cpu-service`, 50 m dla `io-service`), HPA dodaje repliki.

```

1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: cpu-service-hpa
5   namespace: magisterka
6   labels:
7     app: cpu-service
8 spec:
9   scaleTargetRef:
10    apiVersion: apps/v1
11    kind: Deployment
12    name: cpu-service
13   minReplicas: 2
14   maxReplicas: 10
15   metrics:
16   - type: Resource
17     resource:
18       name: cpu
19       target:
20         type: Utilization
21         averageUtilization: 50

```

Listing 3.1 Konfiguracja HPA CPU dla `cpu-service` (plik `k8s/cpu-service/hpa-cpu.yaml`)

Strategia CPU jest najczęściej stosowaną i domyślnie rekomendowaną w ekosystemie *Kubernetes*. Jej skuteczność zależy od tego, czy zużycie CPU koreluje z obciążeniem aplikacji.

3.3.2 HPA Memory (Resource Metric)

Strategia oparta na zużyciu pamięci — HPA monitoruje średnie zużycie pamięci jako procent *resource request*. Wartość progowa: 70%. Strategia ta jest rzadziej stosowana, ponieważ większość aplikacji alokuje pamięć w sposób nieelastyczny (po uruchomieniu lub przy pierwszym żądaniu), co sprawia, że metryka pamięci nie odzwierciedla bieżącego obciążenia.

```

1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: cpu-service-hpa-memory
5   namespace: magisterka
6   labels:
7     app: cpu-service
8     hpa-strategy: memory
9 spec:
10  scaleTargetRef:
11    apiVersion: apps/v1
12    kind: Deployment
13    name: cpu-service
14  minReplicas: 2
15  maxReplicas: 10
16  metrics:
17    - type: Resource
18      resource:
19        name: memory
20        target:
21          type: Utilization
22          averageUtilization: 70

```

Listing 3.2 Konfiguracja HPA Memory dla `cpu-service` (plik `k8s/cpu-service/hpa-memory.yaml`)

Hipoteza badawcza przewiduje, że HPA Memory będzie nieskuteczne dla obu profili obciążeniowych — ani CPU-bound, ani I/O-bound nie powinny generować istotnych wahań zużycia pamięci skorelowanych z ruchem.

3.3.3 HPA Custom RPS (Pods Metric)

Strategia oparta na niestandardowej metryce `http_requests_per_second` — liczbie żądań HTTP na sekundę na Pod. Metryka jest dostarczana przez *prometheus-adapter*, który transformuje liczniki *Prometheusa* (`cpu_service_http_requests_total`, `io_service_http_requests`) na chwilową wartość RPS za pomocą funkcji `rate()` [1m]. Wartość progowa: 100 żądań na sekundę na Pod.

```

1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: cpu-service-hpa-custom
5   namespace: magisterka
6   labels:
7     app: cpu-service
8     hpa-strategy: custom-rps
9 spec:
10  scaleTargetRef:
11    apiVersion: apps/v1

```

```
12     kind: Deployment
13     name: cpu-service
14     minReplicas: 2
15     maxReplicas: 10
16     metrics:
17     - type: Pods
18       pods:
19         metric:
20           name: http_requests_per_second
21         target:
22           type: AverageValue
23           averageValue: "100"
```

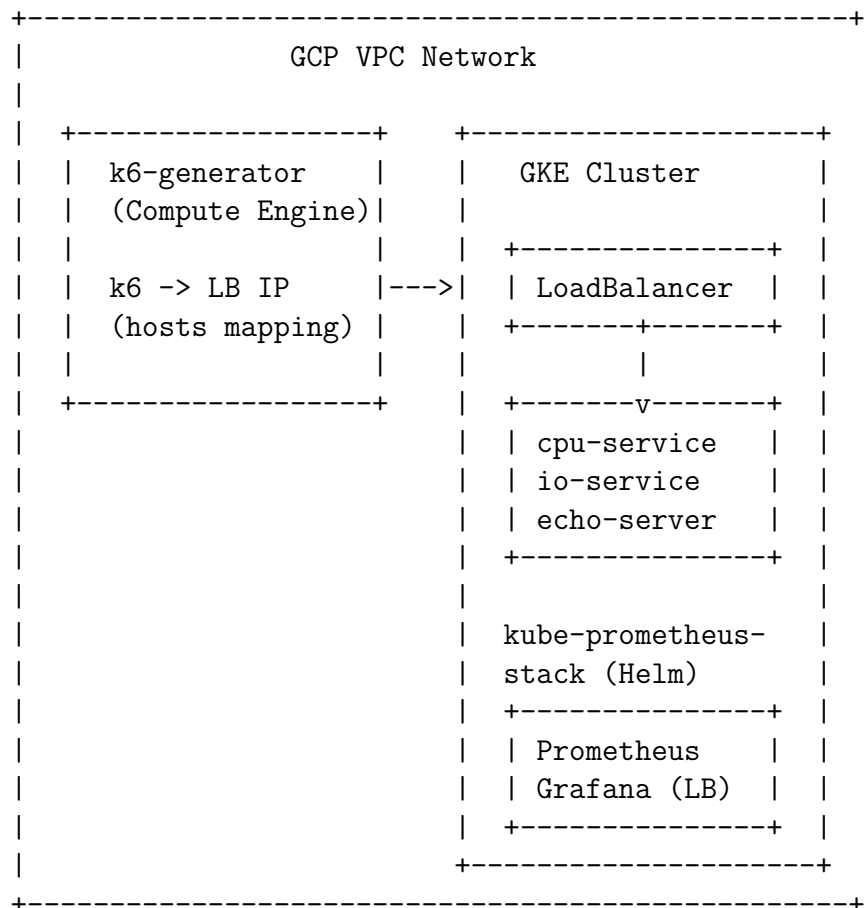
Listing 3.3 Konfiguracja HPA Custom RPS (plik `k8s/cpu-service/hpa-custom.yaml`)

Strategia RPS jest bezpośrednio powiązana z obciążeniem żadaniami — niezależnie od tego, czy żądania są CPU-intensywne, czy I/O-intensywne, liczba żądań na sekundę rośnie proporcjonalnie do ruchu. Dzięki temu HPA Custom RPS powinna być skuteczna dla obu profili obciążeniowych.

3.4 Architektura wdrożeniowa w GCP

3.4.1 Topologia sieciowa GCP

System badawczy jest w całości wdrożony w środowisku *Google Cloud Platform* (GCP). Wszystkie komponenty znajdują się w jednej sieci *Virtual Private Cloud* (VPC), co zapewnia niskie opóźnienia i bezpieczną komunikację wewnętrzną:



Rysunek 3.1 Topologia systemu badawczego w GCP — wszystkie komponenty w jednej sieci VPC

3.4.2 GKE Cluster — klaster Kubernetes

Klaster GKE jest centrum systemu badawczego. Wszystkie aplikacje, HPA, monitoring i *prometheus-adapter* działają jako zasoby *Kubernetes* w namespace *magisterka*. Klaster składa się z węzłów typu *e2-standard-4* (4 vCPU, 16 GB RAM), które zapewniają wystarczające zasoby dla maksymalnie 20 Podów aplikacyjnych (przy *maxReplicas=10* dla obu serwisów) plus Podów monitorujących.

3.4.3 GCP LoadBalancer — ekspozycja serwisów

Każdy serwis aplikacyjny (*cpu-service*, *io-service*, *echo-server*) jest eksponowany jako *Service* typu *LoadBalancer*. GKE automatycznie tworzy zewnętrzny *GCP LoadBalancer* (Layer 4, TCP) i przydziela mu publiczny adres IP. Ruch z maszyny *k6-generator* trafia bezpośrednio na te adresy IP.

Dodatkowo, *Grafana* (część *kube-prometheus-stack*) jest również eksponowana przez *LoadBalancer*, co umożliwi obserwację eksperymentów w czasie rzeczywistym z dowolnej przeglądarki.

3.4.4 k6-generator — maszyna generująca obciążenie

Dedykowana maszyna *Compute Engine* (k6-generator) w tej samej sieci VPC uruchamia skrypty *k6*. Mapowanie nazw serwisów na adresy IP *LoadBalancer* odbywa się przez mechanizm `options.hosts` w skryptach *k6* — bez konieczności konfiguracji DNS:

```
1 export const options = {  
2   hosts: {  
3     'cpu-service': '34.118.71.122',    // GCP LoadBalancer IP  
4     'io-service': '35.204.93.45',     // GCP LoadBalancer IP  
5   },  
6   scenarios: { ... }  
7 };
```

Listing 3.4 Mapowanie hostów w skrypcie *k6* — zamiast DNS

To rozwiązanie eliminuje potrzebę konfiguracji serwera DNS, rekordów `/etc/hosts` czy tunelowania przez `kubectl port-forward`. Ruch sieciowy między k6-generator a Podami w GKE przechodzi przez GCP LoadBalancer, który dystrybuuje go do zdrowych Podów.

3.4.5 Rejestr obrazów kontenerowych

Obrazy Docker są przechowywane w *GCP Artifact Registry*, skąd są pobierane przez GKE podczas wdrażania. Eliminuje to konieczność ręcznego importowania obrazów do *container runtime* na każdym węźle — GKE automatycznie pobiera obrazy z rejestru na podstawie pola `image` w *Deployment*.

3.5 Infrastruktura monitorująca

3.5.1 kube-prometheus-stack

Pakiet *kube-prometheus-stack* jest instalowany przez *Helm* i dostarcza kompletny stos monitorujący:

- **Prometheus** — zbiera metryki infrastrukturalne (przez `kube-state-metrics`, `node-exporter`, `cAdvisor`) oraz aplikacyjne (z endpointów `/metrics` serwisów) co 15 sekund.
- **Grafana** — udostępnia dashboardy wizualizacyjne, eksponowana publicznie przez GCP LoadBalancer dla obserwacji eksperymentów w czasie rzeczywistym.
- **kube-state-metrics** — generuje metryki o stanie obiektów *Kubernetes*: Deployments, HPA, Pody.
- **node-exporter** — zbiera metryki systemowe z węzłów (CPU, pamięć, dysk, sieć).

Prometheus Operator zarządza cyklem życia *Prometheusa* i automatycznie konfiguruje *scraping* na podstawie obiektów *ServiceMonitor* i *PodMonitor*. Aplikacje badawcze są automatycznie wykrywane dzięki adnotacjom `prometheus.io/scrape: "true"` na Podach.

3.5.2 Grafana — dashboard i eksport danych

Grafana udostępnia autorski dashboard „Magisterka — System Overview” zawierający panele wizualizujące kluczowe metryki systemu w czasie rzeczywistym:

- RPS dla `cpu-service` i `io-service` (panele typu `stat` z miniwykresem).
- Zużycie CPU na Pod (panele typu `timeseries`).
- Zużycie pamięci na Pod.
- Liczba replik per *Deployment*.
- Opóźnienia (*latency*): p50, p95, p99.
- Liczba żądań w toku (*in-flight*).
- Stan HPA (pożądana vs. aktualna liczba replik).

Dla każdego z 8 scenariuszy testowych eksportowanych jest 5 zestawów danych CSV z *Grafany*:

1. `replicas.csv` — liczba replik HPA w czasie.
2. `cpu_mcpu.csv` — zużycie CPU per Pod w milijednostkach (mCPU).
3. `memory_mb.csv` — zużycie pamięci per Pod w megabajtach.
4. `rps.csv` — przepustowość (RPS) w czasie.
5. `latency_ms.csv` — opóźnienia odpowiedzi w czasie (p50, p95, p99).

Eksporty te, wraz z wynikami *k6* (JSON + tekstowe podsumowanie), stanowią kompletny zbiór danych dla każdego scenariusza.

3.5.3 Prometheus Adapter

Prometheus Adapter (wersja v0.12.0) implementuje *Custom Metrics API* w *Kubernetes*, umożliwiając HPA odczytywanie metryk z *Prometheusa*. Konfiguracja adaptera (przechowywana w `ConfigMap prometheus-adapter-config`) definiuje cztery reguły mapowania:

1. `cpu_service_http_requests_total` → `http_requests_per_second` (przez `rate()` [1m]).
2. `io_service_http_requests_total` → `http_requests_per_second` (przez `rate()` [1m]).
3. `cpu_service_http_requests_inprogress` → `http_requests_inflight` (wartość chwilowa).
4. `io_service_http_requests_inprogress` → `http_requests_inflight` (wartość chwilowa).

Reguły mapowania wykorzystują mechanizm `<<.LabelMatchers>>` i `<<.GroupBy>>` do dynamicznego filtrowania po namespace i Podzie, co jest wymagane przez *Custom Metrics API*.

3.6 Topologia zasobów Kubernetes

Wszystkie komponenty systemu są wdrażane w pojedynczym *namespace* `magisterka`. Topologia obejmuje:

- **3 Deploymenty** aplikacyjne: `cpu-service`, `io-service`, `echo-server` (plus opcjonalnie `memory-service`).
- **3 Serwisy** typu `LoadBalancer`: każdy kieruje ruch na port 80 do `targetPort 8000` odpowiedniego *Deploymentu*, eksponując serwis przez GCP `LoadBalancer`.
- **6 HPA** (po 3 na serwis badawczy): pliki `hpa-cpu.yaml`, `hpa-memory.yaml`, `hpa-custom.yaml`.
- **Monitoring**: *kube-prometheus-stack* (Prometheus + Grafana + `kube-state-metrics` + `node-exporter`) + `prometheus-adapter`.

W środowisku testowym ruch z *k6* do serwisów jest kierowany przez GCP `LoadBalancer` na podstawie mapowania `options.hosts`. Opóźnienie sieciowe między maszyną `k6-generator` a Podami w GKE wynosi $\sim 1\text{--}5$ ms (komunikacja wewnątrz tej samej strefy VPC).

3.6.1 Zasoby przydzielone serwisom

Serwis	CPU request	CPU limit	Memory request / limit
<code>cpu-service</code>	250 m	1000 m	256 Mi / 512 Mi
<code>io-service</code>	100 m	500 m	128 Mi / 256 Mi
<code>echo-server</code>	50 m	200 m	64 Mi / 128 Mi
<code>memory-service</code>	200 m	500 m	256 Mi / 1024 Mi

Tabela 3.2 Zasoby przydzielone serwisom (*resource requests* i *limits*)

Różnica w przydziale zasobów między `cpu-service` a `io-service` odzwierciedla odmienne profile obciążeniowe: `cpu-service` potrzebuje więcej CPU (250 m vs. 100 m) i pamięci (256 Mi vs. 128 Mi) ze względu na operacje przetwarzania obrazu.

3.7 Plan eksperymentów

3.7.1 Scenariusze testowe

Eksperyment obejmuje 8 scenariuszy uruchamianych sekwencyjnie:

1. `baseline-cpu` — CPU-bound, brak HPA, 1 replika.
2. `baseline-io` — I/O-bound, brak HPA, 1 replika.
3. `cpu-cpu` — CPU-bound + HPA CPU (50%).
4. `cpu-memory` — CPU-bound + HPA Memory (70%).
5. `cpu-custom` — CPU-bound + HPA Custom RPS (100/Pod).

6. `io-cpu` — I/O-bound + HPA CPU (50%).
7. `io-memory` — I/O-bound + HPA Memory (70%).
8. `io-custom` — I/O-bound + HPA Custom RPS (100/Pod).

3.7.2 Procedura eksperymentalna

Każdy scenariusz jest wykonywany według ściśle określonej procedury:

1. **Przełączenie HPA** — usunięcie wszystkich istniejących HPA dla docelowego serwisu i nałożenie pliku HPA odpowiadającego testowanej strategii (przez `kubectl apply`).
2. **Reset Deploymentu** — skalowanie *Deploymentu* do 1 repliki i oczekiwanie na gotowość (`rollout status`).
3. **Uruchomienie testu k6** — wykonanie skryptu obciążeniowego z maszyny `k6-generator`, ruch kierowany na adres IP GCP LoadBalancer przez mapowanie `options.hosts`.
4. **Zapis wyników** — wyniki k6 (JSON + tekstowe podsumowanie) zapisywane do katalogu `results/<scenariusz>/`, wraz z 5 eksportami CSV z *Grafany*.
5. **Powtórzenie** — kroki 1–4 powtarzane 5-krotnie dla każdego scenariusza.
6. **Cooldown** — po każdym scenariuszu: skalowanie obu serwisów do 1 repliki, usunięcie wszystkich HPA, odczekanie 120 sekund na stabilizację klastra.

3.7.3 Profile obciążeniowe k6

Każdy scenariusz wykorzystuje dedykowany skrypt *k6* dopasowany do profilu obciążeniowego:

CPU-bound (skrypt `cpu-load-test.js`):

- **Scenariusz Fibonacci**: ramp-up od 1 do 50 VU w ciągu 60 sekund, następnie stałe obciążenie 50 VU przez 120 sekund.
- **Scenariusz przetwarzanie obrazu**: stałe 10 VU przez 180 sekund, każde żądanie wysyła losowo wygenerowany obraz PNG (400 × 300 pikseli) do `POST /process`.

I/O-bound (skrypt `io-load-test.js`):

- **Scenariusz stałych zapytań**: ramp-up od 1 do 100 VU w ciągu 120 sekund, stałe obciążenie przez 180 sekund, `delay=200ms`, `steps=3`.
- **Scenariusz stres-test**: stałe 30 VU, `delay` rosnący od 50 ms do 2000 ms w krokach co 120 sekund.
- **Scenariusz wywołań zewnętrznych**: stałe 20 VU, `external_call=true`, żądania do `GET /query`.

Scenariusz łączony (skrypt `combined-scenario.js`):

- **Fibonacci**: 30 VU (ramp-up 30s, steady 180s).

- Przetwarzanie obrazu: 5 VU (stałe, 180 s).
- Zapytania IO: 80 VU (ramp-up 60 s, steady 180 s).

Wszystkie skrypty definiują progi akceptacji (*thresholds*): $p(95) < 5000$ ms dla scenariuszy CPU, $p(95) < 10000$ ms dla scenariuszy IO, oraz `http_req_failed` $< 1\%$ (CPU) / $< 5\%$ (IO).

3.7.4 Metryki badawcze

Dla każdego scenariusza zbierane są następujące metryki:

- **Opóźnienie (*latency*)** — $p(50)$, $p(95)$, $p(99)$ czasu odpowiedzi HTTP (ms).
- **Przepustowość (*throughput*)** — liczba żądań na sekundę (RPS) obsłużonych przez system.
- **Czas do skalowania (*time-to-scale*)** — czas od przekroczenia progu HPA do osiągnięcia docelowej liczby replik.
- **Efektywność skalowania (*scaling efficiency*)** — stosunek zmiany przepustowości do zmiany liczby replik.
- **Stopa błędów (*error rate*)** — odsetek żądań zakończonych kodem HTTP ≥ 400 .
- **Liczba replik w czasie** — przebieg czasowy skalowania (dane z *Prometheusa* przez *Grafana* CSV).
- **Zużycie CPU i pamięci** — średnie i szczytowe zużycie zasobów na Pod (dane z *Grafana* CSV).

3.7.5 Analiza zjawisk anomalnych

W trakcie eksperymentów szczególna uwaga zostanie poświęcona trzem zjawiskom przejściowym, które mają istotny wpływ na zachowanie systemów *SaaS* w środowisku chmurowym:

1. **HPA Lag** (opóźnienie reakcji autoskalera) — opóźnienie ~ 60 – 90 sekund między przekroczeniem progu metryki a faktycznym utworzeniem nowych Podów. Wynika ono z sumy: czasu scrapowania metryk przez *Prometheusa* (15 s) + czasu odświeżenia HPA (15 s) + czasu tworzenia Podów i osiągnięcia gotowości. W tym okresie istniejące Pody są przeciążone, co może prowadzić do degradacji wydajności.
2. **Cold Start Latency** (opóźnienie zimnego startu Podów) — nowo utworzone Pody potrzebują 15–30 sekund na osiągnięcie gotowości (pobranie obrazu z rejestru, uruchomienie kontenera, zaliczenie *readiness probe*). W tym oknie czasowym ruch jest już dystrybuowany do nowych Podów przez *LoadBalancer*, co powoduje podwyższone opóźnienia percentyla P99 lub błędy HTTP 502/503.
3. **CPU Throttling Cascade** (kaskadowe błędy przy *CPU throttlingu*) — gdy Pody osiągają limit CPU podczas gwałtownego wzrostu ruchu, mechanizm *CPU throttling* w jądrze Linux (*CFS bandwidth control*) ogranicza czas procesora. Powoduje to kolejgowanie żądań $\rightarrow$ wzrost czasu odpowiedzi $\rightarrow$ *timeouty* w *k6* (dial: i/o

timeout) → błędy HTTP 502. Zjawisko to ma charakter kaskadowy, ponieważ HPA nie zdążył jeszcze dodać nowych replik (efekt *HPA Lag*), tworząc „spirale śmierci”, która ustępuje dopiero po osiągnięciu gotowości przez nowe Pody.

3.8 Podsumowanie architektury

Zaprojektowany system badawczy umożliwia kontrolowane, powtarzalne eksperymenty porównujące skuteczność trzech strategii HPA dla dwóch profili obciążeniowych w środowisku GCP/GKE. Kluczowe decyzje architektoniczne:

1. **Izolacja zmiennych** — każdy scenariusz testuje dokładnie jedną kombinację (profil $\times$ strategia), przy wyizolowanym generatorze obciążenia (osobna VM *Compute Engine*) i ustabilizowanym klastrze (cooldown).
2. **Powtarzalność** — każdy scenariusz jest powtarzany 5-krotnie, co umożliwia analizę statystyczną i eliminację wpływu anomalii.
3. **Automatyzacja** — cały proces (przełączanie HPA, reset Deploymentów, uruchamianie k6, zbieranie wyników, eksport CSV z Grafany) jest zautomatyzowany przez skrypt `run-tests.sh`.
4. **Odtwarzalność** — wszystkie manifesty *Kubernetes*, skrypty k6 i konfiguracje są przechowywane w repozytorium, co umożliwia odtworzenie środowiska na dowolnym klastrze GKE.
5. **Realizm wdrożeniowy** — wykorzystanie GCP LoadBalancer zamiast `port-forward` oraz dedykowanej maszyny *Compute Engine* dla k6 odzwierciedla rzeczywiste warunki produkcyjne.

Rozdział 4

Implementacja

Rozdział niniejszy opisuje implementację systemu badawczego — strukturę kodu, implementację mikroservisów, konfigurację *Kubernetes*, infrastrukturę monitorującą oraz skrypty automatyzujące. Wszystkie listingi odwołują się do rzeczywistych plików w repozytorium projektu.

4.1 Struktura repozytorium

Kod projektu jest zorganizowany w następującej strukturze katalogów:

```
magisterka/
  apps/
    cpu-service/      # CPU-bound microservice
    io-service/       # I/O-bound microservice
    echo-server/      # Auxiliary echo service
    memory-service/   # Memory-bound microservice (extension)
  k8s/
    base/             # Namespace
    cpu-service/      # Deployment + Service + HPA x3
    io-service/       # Deployment + Service + HPA x3
    echo-service/     # Deployment + Service
    memory-service/   # Deployment + Service + HPA
    monitoring/       # Prometheus Adapter config
    ingress.yaml      # Traefik Ingress (legacy, nieużywany na GKE)
  monitoring/
    dashboards/       # Grafana dashboard JSON
  scripts/
    deploy-k3s.sh     # Skrypt wdrożeniowy (legacy, GKE używa kubectl apply)
    run-tests.sh      # Test orchestration (8 scenarios)
    run-all.sh        # deploy + tests combined (legacy)
  tests/
    k6/               # k6 load test scripts
  docs/
    prd.md            # Product Requirements Document
    docker-compose.yaml # Opcjonalny development (nieużywany w badaniach)
```

4.2 Implementacja mikroservisów

Każdy mikroservis jest zaimplementowany jako niezależna aplikacja *FastAPI* (Python 3.12), konteneryzowana za pomocą *Docker* i uruchamiana przez *Uvicorn* (serwer ASGI).

4.2.1 Wspólne cechy implementacyjne

Wszystkie serwisy dzielą następujące cechy:

- **FastAPI + Uvicorn** — framework *FastAPI* 0.115.6 z serwerem *Uvicorn* 0.34.0. Wybór *FastAPI* podyktowany jest natywną obsługą asynchroniczności (`async/await`), automatyczną generacją dokumentacji OpenAPI oraz lekkością.
- **Dockerfile** — identyczny wzorzec dla wszystkich serwisów: obraz bazowy `python:3.12-slim`, instalacja zależności przez `pip install --no-cache-dir`, aplikacja kopiowana jako pojedynczy plik `main.py`.
- **Prometheus metrics** — automatyczna instrumentacja przez `prometheus-fastapi-instrument` 7.0.0, która wystawia metryki HTTP (liczniki żądań, histogramy czasu odpowiedzi, liczba żądań w toku) na endpointzie `/metrics`.
- **Health check** — endpoint `GET /health` zwracający `{"status": "ok", "service": "..."}` , wykorzystywany przez *liveness* i *readiness probes* w *Kubernetes*.
- **CORS** — middleware `CORSMiddleware` z zezwoleniem na wszystkie źródła (`allow_origins=["*"]`) dla wygody testowania.

```

1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY main.py .
6 EXPOSE 8000
7 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Listing 4.1 Dockerfile dla `cpu-service` (wzorzec wspólny dla wszystkich serwisów)

4.2.2 Budowanie i dystrybucja obrazów

Obrazy Docker są budowane lokalnie i przesyłane do rejestru kontenerów GCP (*Artifact Registry*) za pomocą poleceń `docker build` i `docker push`. Alternatywnie, *Cloud Build* może budować obrazy bezpośrednio w GCP. GKE pobiera obrazy z rejestru automatycznie na podstawie pola `image` w *Deploymencie*:

```

1 spec:
2   containers:
3     - name: cpu-service
4       image: europe-central2-docker.pkg.dev/PROJECT_ID/magisterka/cpu-
         service:latest
5       imagePullPolicy: Always

```

Listing 4.2 Odwołanie do obrazu w GCP Artifact Registry (wzorzec)

4.2.3 CPU-service

cpu-service implementuje dwa endpointy obciążające procesor:

POST /process — przetwarzanie obrazu

Endpoint akceptuje obraz w formacie *multipart form-data*, wykonuje 5-etapowy potok przetwarzania CPU-intensywnego z użyciem biblioteki *Pillow* 11.1.0:

1. Konwersja do skali szarości (`image.convert("L")`).
2. Zastosowanie filtru BLUR.
3. Zastosowanie filtru SHARPEN.
4. Zastosowanie filtru FIND_EDGES.
5. Ekstrakcja kanałów RGB i resize.

Po przetworzeniu, dla każdego kanału obliczana jest entropia Shannona według wzoru:

$$H = - \sum_{i=0}^{255} p_i \log_2(p_i), \quad \text{gdzie } p_i = \frac{\text{histogram}[i]}{\text{total_pixels}} \quad (4.1)$$

```

1  -----
2  def _shannon_entropy(histogram: list[int], total_pixels: int) -> float:
3      """Calculate Shannon entropy from a pixel-value histogram."""
4      entropy = 0.0
5      for count in histogram:
6          if count == 0:
7              continue
8          prob = count / total_pixels
9          entropy -= prob * math.log2(prob)
10     return entropy

```

Listing 4.3 Funkcja `shannon_entropy()` w `cpu-service`

GET /fibonacci — ciąg Fibonacciego

Endpoint oblicza n -tą liczbę Fibonacciego rekurencyjnym algorytmem o złożoności $O(2^n)$. Parametr n jest pobierany z *query string* (domyślnie 25). Wybór rekurencyjnej implementacji jest celowy — algorytm ten jest skrajnie nieefektywny obliczeniowo, co gwarantuje wysokie zużycie CPU przy niewielkiej liczbie żądań.

```

1  ----
2  def _fib_recursive(n: int) -> int:
3      """Compute the n-th Fibonacci number using naive double recursion.
4
5      This is intentionally O(2^n) - the worst possible implementation -
6      to serve as a pure CPU-burn endpoint for scalability experiments.
7      """
8      if n <= 1:
9          return n
10     return _fib_recursive(n - 1) + _fib_recursive(n - 2)

```

Listing 4.4 Rekurencyjna implementacja ciągu Fibonacciego w `cpu-service`

Metryki niestandardowe

cpu-service wystawia dwie niestandardowe metryki *Prometheus*:

- `cpu_service_images_processed_total` — *Counter* zliczający przetworzone obrazy.
- `cpu_service_processing_duration_seconds` — *Histogram* czasu przetwarzania obrazu.

4.2.4 IO-service

io-service implementuje dwa endpointy symulujące operacje I/O:

GET /query — łańcuch opóźnień asynchronicznych

Endpoint wykonuje sekwencję wywołań `asyncio.sleep()` z konfigurowalnymi parametrami:

- `delay` (domyślnie 100 ms) — bazowe opóźnienie pojedynczego kroku.
- `steps` (domyślnie 3) — liczba kroków w łańcuchu.
- `jitter` (domyślnie 0.2) — losowy rozrzut $\pm 20\%$ wokół `delay`.

Całkowity czas odpowiedzi wynosi $\text{steps} \times \text{delay} \times (1 \pm \text{jitter})$, a procesor pozostaje bezczynny podczas oczekiwania (`asyncio.sleep` zwalnia wątek). Dzięki temu zużycie CPU nie koreluje z liczbą żądań — serwis jest czysto *I/O-bound*.

```

1  -----
2  @app.get("/query", response_model=QueryResponse)
3  async def query(
4      delay: int = Query(200, ge=0, le=5000, description="Base delay per step
        in milliseconds"),
5      steps: int = Query(3, ge=1, le=10, description="Number of simulated
        calls in the chain"),
6      external_call: bool = Query(
7          True,
8          description="If true, call /upstream via HTTP; if false, use
        asyncio.sleep",
9      ),
10 ):
11     """Simulate a chain of I/O-bound operations.
12
13     Each step introduces a delay with  $\pm 20\%$  random jitter. When
14     ``external_call=true`` the step performs an HTTP GET to ``/upstream``;
15     otherwise it uses ``asyncio.sleep``.
16
17     Returns timing breakdown for every step plus the aggregate.
18     """
19     _validate_query_params(delay, steps)
20
21     semaphore: asyncio.Semaphore = app.state.concurrency_semaphore
22     http_client: httpx.AsyncClient = app.state.http_client
23
24     # Track concurrency via the gauge - increment once inside the semaphore
25     await semaphore.acquire()

```

```

26     io_service_concurrent_queries.inc()
27
28     try:
29         step_times_ms: list[float] = []
30
31         total_start = time.perf_counter()
32
33         for _step_idx in range(steps):
34             # Apply ±20 % random jitter to the base delay
35             jitter = random.uniform(0.8, 1.2)
36             actual_delay_ms = delay * jitter
37
38             step_start = time.perf_counter()
39
40             if external_call:
41                 # Real HTTP call to the upstream helper endpoint
42                 try:
43                     response = await http_client.get(
44                         f"{_UPSTREAM_BASE_URL}/upstream",
45                         params={"delay": int(actual_delay_ms)},
46                     )
47                     response.raise_for_status()
48                 except httpx.HTTPError as exc:
49                     # If the upstream call fails, still record the timing
50                     # and propagate a meaningful error to the caller
51                     step_elapsed = time.perf_counter() - step_start
52                     step_times_ms.append(round(step_elapsed * 1000, 2))
53                     raise HTTPException(
54                         status_code=502,
55                         detail=f"Upstream call failed at step {_step_idx + 1}: {exc}",
56                     )
57                 step_elapsed = time.perf_counter() - step_start
58                 # Observe upstream duration in the histogram (in seconds)
59                 io_service_upstream_duration_seconds.observe(step_elapsed)

```

Listing 4.5 Endpoint GET /query w io-service — łańcuch opóźnień asynchronicznych

GET /upstream — wywołanie zewnętrzne

Endpoint przekazuje żądanie do echo-server za pomocą `httpx.AsyncClient` z timeoutem 10 sekund i zwraca odpowiedź z dodatkowym opóźnieniem symulującym przetwarzanie. Parametry: `delay` (domyślnie 50 ms), `jitter` (domyślnie 0.1).

Kontrola współbieżności

Serwis wykorzystuje `asyncio.Semaphore(50)` do ograniczenia liczby współbieżnych zapytań, co zapobiega wyczerpaniu zasobów systemowych przy bardzo wysokim obciążeniu.

```

1  -----
2  @asynccontextmanager
3  async def lifespan(app: FastAPI):
4      """Startup: create shared httpx client and concurrency semaphore.
5      Shutdown: close the httpx client."""
6      app.state.http_client = httpx.AsyncClient(
7          timeout=httpx.Timeout(30.0),
8          limits=httpx.Limits(max_keepalive_connections=20, max_connections
                              =100),

```

```

9         )
10        app.state.concurrency_semaphore = asyncio.Semaphore(50)
11        try:
12            yield
13        finally:
14            await app.state.http_client.aclose()

```

Listing 4.6 Inicjalizacja semafora współbieżności w `io-service`

Metryki niestandardowe

`io-service` wystawia trzy niestandardowe metryki:

- `io_service_queries_total` — *Counter* zliczający zapytania.
- `io_service_upstream_duration_seconds` — *Histogram* czasu wywołań zewnętrznych.
- `io_service_concurrent_queries` — *Gauge* liczby aktualnie przetwarzanych zapytań.

4.2.5 Echo-server

`echo-server` jest minimalistycznym serwisem pomocniczym z dwoma endpointami: `GET /health` i `POST /echo`. Endpoint `/echo` zwraca kopię otrzymanego żądania (nagłówki, *body*, parametry), umożliwiając weryfikację poprawnego przekazywania danych przez `io-service` w scenariuszu `external_calls`.

Serwis korzysta z biblioteki `httpx` do opcjonalnych wywołań zwrotnych do `io-service` i rejestruje niestandardowe liczniki `echo_server_echoes_total` oraz `echo_server_upstream_calls_t`.

4.2.6 Memory-service

`memory-service` implementuje pamięciową bazę klucz-wartość (*in-memory cache*) z limitem 10000 wpisów (`MAX_CACHE_ENTRIES`). Udostępnia pełne CRUD dla cache: `GET /cache`, `GET /cache/{key}`, `POST /cache/{key}`, `DELETE /cache/{key}`, `DELETE /cache`.

Endpoint badawczy `POST /cache/fill` wypełnia cache danymi testowymi o kontrolowanym rozmiarze — parametry `sessions` (liczba wpisów, 1–5000) i `size_kb` (rozmiar pojedynczego wpisu w KB, 1–5000). Wartości są generowane przez `random.choices` z alfabetu ASCII, co zapobiega optymalizacji *string-interning* Pythona.

```

1  -----
2  def _generate_value(size_kb: int) -> str:
3      """Generate a string of approximately *size_kb* kilobytes using random
4          ASCII letters.
5
6          Uses ``random.choices`` to avoid Python's string-interning optimisation
7          that would make ``'x' * N`` consume less memory than expected.
8          """
9      length = size_kb * 1024
10     return "".join(random.choices(string.ascii_letters, k=length))

```

Listing 4.7 Generowanie wartości testowych w `memory-service`

Endpoint `GET /stats` zwraca statystyki cache: liczbę wpisów, szacowany rozmiar, liczbę trafień/chybień, współczynnik trafień (*hit ratio*) oraz średnią liczbęostępów.

4.3 Konfiguracja Kubernetes

4.3.1 Namespace i struktura zasobów

Wszystkie zasoby *Kubernetes* są wdrażane w dedykowanym *namespace* *magisterka*. Plik `k8s/base/namespace.yaml` definiuje ten namespace.

4.3.2 Deploymenty

Każdy *Deployment* definiuje:

- `replicas`: 2 — początkowa liczba replik.
- `imagePullPolicy`: `Always` — obrazy są pobierane z GCP Artifact Registry przy każdym wdrożeniu.
- `livenessProbe` i `readinessProbe` — sprawdzanie `GET /health` co 10 sekund, z początkowym opóźnieniem 5 sekund.
- `resources` — *requests* i *limits* CPU i pamięci (Tabela 3.2).

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: cpu-service
5   namespace: magisterka
6   labels:
7     app: cpu-service
8     tier: backend
9 spec:
10   replicas: 2
11   selector:
12     matchLabels:
13       app: cpu-service
14   template:
15     metadata:
16       labels:
17         app: cpu-service
18         tier: backend
19     spec:
20       containers:
21       - name: cpu-service
22         image: cpu-service:latest
23         imagePullPolicy: IfNotPresent
24         ports:
25         - containerPort: 8000
26       resources:
27         requests:
28           cpu: 250m
29           memory: 256Mi
30         limits:
31           cpu: 1000m
32           memory: 512Mi
33       livenessProbe:
34         httpGet:
35           path: /health
36           port: 8000
```

```
37         initialDelaySeconds: 5
38         periodSeconds: 10
39     readinessProbe:
40         httpGet:
41             path: /health
42             port: 8000
43         initialDelaySeconds: 5
44         periodSeconds: 10
```

Listing 4.8 Deployment cpu-service (plik k8s/cpu-service/deployment.yaml)

4.3.3 HPA

Dla cpu-service i io-service zdefiniowano po trzy warianty HPA, różniące się wyłącznie typem metryki. Każdy HPA używa API autoscaling/v2 i ma zakres 2–10 replik.

HPA CPU

```
1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4      name: cpu-service-hpa
5      namespace: magisterka
6      labels:
7          app: cpu-service
8  spec:
9      scaleTargetRef:
10         apiVersion: apps/v1
11         kind: Deployment
12         name: cpu-service
13     minReplicas: 2
14     maxReplicas: 10
15     metrics:
16     - type: Resource
17       resource:
18         name: cpu
19         target:
20             type: Utilization
21             averageUtilization: 50
```

Listing 4.9 HPA CPU dla cpu-service (plik k8s/cpu-service/hpa-cpu.yaml)

HPA Memory

```
1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4      name: cpu-service-hpa-memory
5      namespace: magisterka
6      labels:
7          app: cpu-service
8          hpa-strategy: memory
9  spec:
10     scaleTargetRef:
11         apiVersion: apps/v1
12         kind: Deployment
```

```
13     name: cpu-service
14   minReplicas: 2
15   maxReplicas: 10
16   metrics:
17   - type: Resource
18     resource:
19       name: memory
20     target:
21       type: Utilization
22       averageUtilization: 70
```

Listing 4.10 HPA Memory dla cpu-service (plik k8s/cpu-service/hpa-memory.yaml)

HPA Custom RPS

```
1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4    name: cpu-service-hpa-custom
5    namespace: magisterka
6    labels:
7      app: cpu-service
8      hpa-strategy: custom-rps
9  spec:
10    scaleTargetRef:
11      apiVersion: apps/v1
12      kind: Deployment
13      name: cpu-service
14    minReplicas: 2
15    maxReplicas: 10
16    metrics:
17    - type: Pods
18      pods:
19        metric:
20          name: http_requests_per_second
21        target:
22          type: AverageValue
23          averageValue: "100"
```

Listing 4.11 HPA Custom RPS dla cpu-service (plik k8s/cpu-service/hpa-custom.yaml)

4.3.4 Services — LoadBalancer w GKE

W odróżnieniu od typowego środowiska deweloperskiego (gdzie stosuje się `ClusterIP` z `port-forward`), wszystkie serwisy aplikacyjne są typu `LoadBalancer`. GKE automatycznie tworzy zewnętrzny *GCP LoadBalancer* (Layer 4, TCP) i przydziela mu publiczny adres IP. Ruch z maszyny `k6-generator` trafia bezpośrednio na ten adres IP, co odzwierciedla warunki produkcyjne.

Plik `k8s/ingress.yaml` (konfiguracja *Traefik Ingress* z routowaniem przez hosty `*.k3s.local`) pozostaje w repozytorium jako artefakt historyczny, ale **nie jest używany** w środowisku GKE — komunikacja odbywa się bezpośrednio przez GCP LoadBalancer.

4.4 Infrastruktura monitorująca

4.4.1 kube-prometheus-stack — instalacja przez Helm

Pakiet *kube-prometheus-stack* jest instalowany przez *Helm* — menedżera pakietów dla *Kubernetes*. Instalacja odbywa się trzema poleceniami:

```

1 # Dodanie repozytorium Helm
2 helm repo add prometheus-community \
3   https://prometheus-community.github.io/helm-charts
4 helm repo update
5
6 # Instalacja w namespace magisterka
7 helm install monitoring prometheus-community/kube-prometheus-stack \
8   --namespace magisterka --create-namespace \
9   --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=
10  false \
   --set grafana.service.type=LoadBalancer

```

Listing 4.12 Instalacja *kube-prometheus-stack* przez Helm

Parametr `grafana.service.type=LoadBalancer` powoduje, że *Grafana* jest ekspozowana przez GCP LoadBalancer, co umożliwia dostęp do dashboardów z dowolnej przeglądarki bez konieczności tunelowania.

Pakiet instaluje następujące komponenty:

- **Prometheus** — centralny system zbierania metryk, automatycznie skonfigurowany do wykrywania *ServiceMonitor* i *PodMonitor* w namespace `magisterka`.
- **Grafana** — platforma wizualizacji z predefiniowanymi dashboardami *Kubernetes*.
- **kube-state-metrics** — generuje metryki o stanie obiektów *Kubernetes* (Deployments, HPA, Pody).
- **node-exporter** — zbiera metryki systemowe z węzłów.
- **Prometheus Operator** — zarządza cyklem życia *Prometheusa*.

4.4.2 Prometheus Adapter

Prometheus Adapter v0.12.0 implementuje *Custom Metrics API*. W odróżnieniu od *kube-prometheus-stack*, adapter jest wdrażany przez osobne manifesty YAML, ponieważ nie jest częścią pakietu *Helm*. Konfiguracja w `ConfigMap` definiuje reguły *PromQL* dla metryk RPS i *in-flight* dla obu serwisów badawczych:

```

1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: prometheus-adapter-config
5   namespace: magisterka
6   labels:
7     app: prometheus-adapter
8     tier: monitoring
9 data:
10  config.yaml: |
11    rules:
12      - seriesQuery: 'cpu_service_http_requests_total{namespace!="",pod
13                    !=""}'

```

```

13     resources:
14         overrides:
15             namespace: {resource: "namespace"}
16             pod: {resource: "pod"}
17     name:
18         matches: "cpu_service_http_requests_total"
19         as: "http_requests_per_second"
20     metricsQuery: 'sum(rate(cpu_service_http_requests_total{<<.
21         LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
22 - seriesQuery: 'cpu_service_http_requests_inprogress{namespace!="",
23     pod!=""}'
24     resources:
25         overrides:
26             namespace: {resource: "namespace"}
27             pod: {resource: "pod"}
28     name:
29         matches: "cpu_service_http_requests_inprogress"
30         as: "http_requests_inflight"
31     metricsQuery: 'sum(cpu_service_http_requests_inprogress{<<.
32         LabelMatchers>>}) by (<<.GroupBy>>)'
33 - seriesQuery: 'io_service_http_requests_total{namespace!="",pod!=""}
34     ,
35     resources:
36         overrides:
37             namespace: {resource: "namespace"}
38             pod: {resource: "pod"}
39     name:
40         matches: "io_service_http_requests_total"
41         as: "http_requests_per_second"
42     metricsQuery: 'sum(rate(io_service_http_requests_total{<<.
43         LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
44 - seriesQuery: 'io_service_http_requests_inprogress{namespace!="",pod
45     !=""}'
46     resources:
47         overrides:
48             namespace: {resource: "namespace"}
49             pod: {resource: "pod"}
50     name:
51         matches: "io_service_http_requests_inprogress"
52         as: "http_requests_inflight"
53     metricsQuery: 'sum(io_service_http_requests_inprogress{<<.
54         LabelMatchers>>}) by (<<.GroupBy>>)'
55 - seriesQuery: '{__name__=~"^container_.*",namespace!="",pod!=""}'
56     resources:
57         overrides:
58             namespace: {resource: "namespace"}
59             pod: {resource: "pod"}
60     name:
61         matches: "^container_(.*)$"
62         as: ""
63     metricsQuery: 'sum(<<.Series>>{<<.LabelMatchers>>,container!="POD",
64         container!=""}) by (<<.GroupBy>>)'

```

Listing 4.13 Konfiguracja *Prometheus Adapter* — reguły mapowania (plik `k8s/monitoring/prometheus-adapter-config.yaml`)


```

13 info "❗Przeczenie HPA dla ${service} → strategia: ${strategy}"
14
15 # ❗Usu wszystkie istniejące HPA dla tego serwisu
16 local hpa_names
17 hpa_names=$((${KUBECTL} -n "${NAMESPACE}" get hpa -o name 2>/dev/null |
18   grep "${service}" || true)
19 if [ -n "${hpa_names}" ]; then
20   for hpa in ${hpa_names}; do
21     ${KUBECTL} -n "${NAMESPACE}" delete "${hpa}" --ignore-not-found
22     =true
23   done
24   ok " ☑ Usunieto poprzednie HPA dla ${service}"
25 fi
26
27 # ❗Nakadanie wybranej strategii
28 case "${strategy}" in
29   cpu)
30     ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-cpu.yaml"
31     ok " ❗ Naono HPA CPU"
32     ;;
33   memory)
34     ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-memory.yaml"
35     ok " ❗ Naono HPA Memory"
36     ;;
37   custom)
38     ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-custom.yaml"
39     ok " ❗ Naono HPA Custom (RPS)"
40     ;;
41   none)
42     ok "   Brak HPA (baseline)"
43     ;;
44   *)
45     err "   Nieznana strategia: ${strategy}"
46     return 1
47     ;;
48 esac
49
50 sleep 2
51 }

```

Listing 4.14 Funkcja `switch_hpa()` — przełączanie strategii HPA (plik `scripts/run-tests.sh`)

```

1  # --- Cooldown pomiędzy scenariuszami (D7)
2  -----
3  cooldown() {
4      info "==== COOLDOWN: Stabilizacja klastra (120s) ====="
5
6      # 1. Skaluj oba serwisy do 1 repliki
7      info "Skalowanie cpu-service i io-service → 1 replika..."
8      ${KUBECTL} -n "${NAMESPACE}" scale deployment cpu-service io-service --
9          replicas=1 2>/dev/null || true
10
11     # 2. Usuń wszystkie aktywne HPA
12     info "Usuwanie wszystkich HPA w namespace ${NAMESPACE}..."
13     ${KUBECTL} -n "${NAMESPACE}" delete hpa --all --ignore-not-found=true
14         2>/dev/null || true

```

```

14     # 3. Czekaj 120s na ęstabilizacj
15     info "Oczekiwanie 120s na ęstabilizacj klastra..."
16     sleep 120
17
18     ok "Cooldown ńzakoczony - klaster ustabilizowany"
19     echo ""
20 }

```

Listing 4.15 Funkcja `cooldown()` — stabilizacja klastra między scenariuszami (plik `scripts/run-tests.sh`)

4.5.2 Uwaga o skryptach legacy

W repozytorium znajdują się również skrypty `deploy-k3s.sh` oraz `run-all.sh`, które były przewidziane dla lokalnego środowiska *k3s*. Skrypty te **nie są wykorzystywane** w środowisku GKE — wdrożenie na GKE odbywa się przez `kubectl apply` dla manifestów *Kubernetes* oraz `helm install` dla *kube-prometheus-stack*. Skrypty pozostają w repozytorium jako artefakty fazy projektowej.

4.6 Scenariusze testowe k6

Każdy skrypt *k6* definiuje scenariusze obciążeniowe, progi akceptacji (*thresholds*) oraz mapowanie hostów (`options.hosts`) dla adresów IP GCP LoadBalancer:

4.6.1 `cpu-load-test.js`

```

1  ---
2  export const options = {
3    scenarios: {
4      // Scenario A - Fibonacci ramp-up
5      fibonacci: {
6        executor: 'ramping-vus',
7        startVUs: 1,
8        stages: [
9          { duration: '60s', target: 50 },    // ramp-up
10         { duration: '120s', target: 50 },    // steady
11         { duration: '30s', target: 0 },      // ramp-down
12       ],
13       exec: 'fibonacci',
14       gracefulRampDown: '30s',
15     },
16
17     // Scenario B - Image processing at constant load
18     process_image: {
19       executor: 'constant-vus',
20       vus: 10,
21       duration: '180s',
22       exec: 'processImage',
23       gracefulRampDown: '30s',
24     },
25   },
26
27   thresholds: {
28     // 95th percentile of request duration < 5 s

```



```

29     'http_req_duration': ['p(95)<5000'],
30     // Less than 1 % failed requests
31     'http_req_failed': ['rate<0.01'],
32   },
33 };

```

Listing 4.16 Scenariusze i progi w cpu-load-test.js

4.6.2 io-load-test.js

```

1  ---+---
2  export const options = {
3    scenarios: {
4      // Scenario A - Constant delay, ramp-up VUs
5      constant_query: {
6        executor: 'ramping-vus',
7        startVUs: 1,
8        stages: [
9          { duration: '120s', target: 100 }, // ramp-up
10         { duration: '180s', target: 100 }, // steady
11         { duration: '60s', target: 0 },    // ramp-down
12       ],
13       exec: 'constantQuery',
14       gracefulRampDown: '30s',
15     },
16
17     // Scenario B - Increasing delay (stress test)
18     stress_test: {
19       executor: 'constant-vus',
20       vus: 30,
21       duration: '120s',
22       exec: 'stressTest',
23       gracefulRampDown: '30s',
24     },
25
26     // Scenario C - External calls
27     external_calls: {
28       executor: 'constant-vus',
29       vus: 20,
30       duration: '120s',
31       exec: 'externalCalls',
32       gracefulRampDown: '30s',
33     },
34   },
35
36   thresholds: {
37     // 95th percentile of request duration < 10 s
38     'http_req_duration': ['p(95)<10000'],
39     // Less than 5 % failed requests
40     'http_req_failed': ['rate<0.05'],
41   },
42 };

```

Listing 4.17 Scenariusze i progi w io-load-test.js

4.6.3 combined-scenario.js

```

1 -----
2 export const options = {
3   scenarios: {
4     cpu_fibonacci: {
5       executor: 'ramping-vus',
6       startVUs: 0,
7       stages: [
8         { duration: '60s', target: 30 },
9         { duration: '180s', target: 30 },
10        { duration: '60s', target: 0 },
11      ],
12      exec: 'fibonacci',
13      gracefulRampDown: '30s',
14    },
15    cpu_processing: {
16      executor: 'constant-vus',
17      vus: 5,
18      duration: '300s',
19      exec: 'processImage',
20      startTime: '30s',
21      gracefulRampDown: '30s',
22    },
23    io_queries: {
24      executor: 'ramping-vus',
25      startVUs: 0,
26      stages: [
27        { duration: '60s', target: 80 },
28        { duration: '180s', target: 80 },
29        { duration: '60s', target: 0 },
30      ],
31      exec: 'ioQuery',
32      gracefulRampDown: '30s',
33    },
34  },
35
36  thresholds: {
37    'http_req_duration{scenario:cpu_fibonacci}': ['p(95)<5000'],
38    'http_req_duration{scenario:cpu_processing}': ['p(95)<10000'],
39    'http_req_duration{scenario:io_queries}': ['p(95)<10000'],
40    'http_req_failed': ['rate<0.05'],
41  },
42 };

```

Listing 4.18 Scenariusze i progi w combined-scenario.js

4.7 Podsumowanie implementacji

Zaimplementowany system badawczy składa się z czterech mikroserwisów (`cpu-service`, `io-service`, `echo-server`, `memory-service`), infrastruktury monitorującej opartej na *kube-prometheus-stack* (Helm) oraz zestawu skryptów automatyzujących testy. Kluczowe cechy implementacji:

- **Środowisko GCP/GKE** — całość infrastruktury działa na zarządzanym klastrze *Google Kubernetes Engine*, z serwisami eksponowanymi przez GCP LoadBalancer i maszyną *Compute Engine* jako generatorem obciążenia.

- **Monitoring przez kube-prometheus-stack** — instalacja przez *Helm* zapewnia spójny, automatycznie skonfigurowany stos monitorujący bez ręcznych manifestów.
- **Pełna automatyzacja** — skrypt `run-tests.sh` orkiestruje wszystkie 8 scenariuszy z 5-krotnym powtórzeniem, z automatycznym przełączaniem HPA i cooldownem.
- **Powtarzalność** — każdy scenariusz testowy jest uruchamiany 5-krotnie, z cooldownem między scenariuszami.
- **Odtwarzalność** — wszystkie manifesty są deklaratywne i mogą być odtworzone na dowolnym klastrze GKE.
- **Obserwowalność** — metryki zbierane przez *Prometheusa* i wizualizowane w *Grafanie* (dostępnej publicznie przez LoadBalancer), z eksportem 5 zestawów CSV na scenariusz.

Rozdział 5

Badania eksperymentalne i analiza wyników

Rozdział niniejszy opisuje metodykę badań eksperymentalnych oraz przedstawia analizę wyników. Badania zostały zaprojektowane w celu empirycznej weryfikacji hipotezy badawczej: skuteczność strategii HPA zależy od charakterystyki obciążeniowej aplikacji. Wszystkie eksperymenty są przeprowadzane w środowisku *Google Cloud Platform* (GCP), z klastrem *Google Kubernetes Engine* (GKE) jako platformą badawczą.

5.1 Metodyka badań

5.1.1 Środowisko testowe — GKE

Badania są prowadzone wyłącznie w środowisku chmurowym *Google Cloud Platform*, składającym się z następujących komponentów:

- **GKE Cluster** — zarządzany klaster *Kubernetes* na maszynach **e2-standard-4** (4 vCPU, 16 GB RAM). Klaster działa w konfiguracji VPC-native, z automatycznym skalowaniem węzłów (*cluster autoscaler*). Wszystkie zasoby badawcze są wdrażane w namespace **magisterka**.
- **k6-generator** — dedykowana maszyna *Compute Engine* w tej samej sieci VPC co klaster GKE. Maszyna jest wyłącznym hostem dla narzędzia *k6*, generującego obciążenie testowe. Współdzielenie sieci VPC zapewnia niskie opóźnienie ($\sim 1\text{--}5$ ms) między generatorem obciążenia a serwisami aplikacyjnymi, a separacja maszyn zapobiega zakłócaniu wyników przez konkurencję o zasoby.
- **GCP LoadBalancer** — każdy serwis aplikacyjny (**cpu-service**, **io-service**, **echo-server**) jest eksponowany jako **Service** typu **LoadBalancer**. GKE automatycznie tworzy zewnętrzny *GCP LoadBalancer* (Layer 4, TCP) i przydziela mu publiczny adres IP. Ruch z **k6-generator** trafia bezpośrednio na te adresy IP.
- **kube-prometheus-stack** — stos monitorujący (*Prometheus*, *Grafana*, *kube-state-metrics*, *node-exporter*) instalowany przez *Helm*. *Grafana* jest eksponowana przez **LoadBalancer**, co umożliwia obserwację eksperymentów w czasie rzeczywistym.

5.1.2 Procedura eksperymentalna

Każdy z 8 scenariuszy badawczych (2 bazowe + 6 z macierzy 2×3) jest wykonywany według ściśle określonej procedury:

1. **Przygotowanie** — usunięcie wszystkich istniejących HPA dla docelowego serwisu, nałożenie pliku HPA odpowiadającego testowanej strategii (przez `kubectl apply` z poziomu `k6-generator`).
2. **Reset** — skalowanie *Deploymentu* do 1 repliki (`kubectl scale --replicas=1`), oczekiwanie na gotowość (`rollout status`).
3. **Obciążenie** — uruchomienie skryptu *k6* odpowiedniego dla profilu obciążeniowego (`cpu-load-test.js` lub `io-load-test.js`). Ruch jest kierowany na adresy IP *GCP LoadBalancer* przez mechanizm `options.hosts` w skryptach *k6* — bez konfiguracji DNS ani tunelowania `port-forward`.
4. **Zapis** — wyniki *k6* (JSON + tekstowe podsumowanie) oraz metadane uruchomienia zapisywane do `results/<scenariusz>/run-<i>/`. Dodatkowo, z *Grafany* eksportowanych jest 5 zestawów danych CSV dla każdego powtórzenia.
5. **Powtórzenie** — kroki 1–4 powtarzane 5-krotnie dla każdego scenariusza.
6. **Stabilizacja** — po każdym scenariuszu: skalowanie obu serwisów do 1 repliki, usunięcie wszystkich HPA, odczekanie 120 sekund (*cooldown*).

5.1.3 Zbierane metryki

Dla każdego powtórzenia każdego scenariusza zbierane są następujące dane:

- **Metryki HTTP z k6**: liczba żądań, RPS (*throughput*), percentyle czasu odpowiedzi (p50, p95, p99), minimalny/maksymalny/średni czas odpowiedzi, liczba błędów HTTP (≥ 400).
- **Eksporty CSV z Grafany** — dla każdego scenariusza eksportowanych jest 5 zestawów danych szeregów czasowych:
 1. `replicas.csv` — liczba replik HPA w funkcji czasu (źródło: `kube_deployment_spec_replicas`).
 2. `cpu_mcpu.csv` — zużycie CPU per Pod w milijednostkach (źródło: `container_cpu_usage_seconds_total`).
 3. `memory_mb.csv` — zużycie pamięci per Pod w megabajtach (źródło: `container_memory_working_set_bytes`).
 4. `rps.csv` — przepustowość w funkcji czasu (źródło: `rate(http_requests_total[30s])`).
 5. `latency_ms.csv` — percentyle opóźnień w funkcji czasu: p50, p95, p99 (źródło: `histogram_quantile()` na `http_request_duration_seconds_bucket`).
- **Metadane**: scenariusz, strategia HPA, numer powtórzenia, timestamp, kod wyjścia k6.

5.1.4 Metryki badawcze — definicje operacyjne

Na podstawie surowych danych obliczane są następujące metryki badawcze:

1. **Średnia przepustowość** (T) — średnia liczba żądań na sekundę w trakcie testu, obliczana jako `total_requests/duration`.
2. **Opóźnienie p95** (L_{95}) — 95. percentyl czasu odpowiedzi HTTP (ms). Reprezentuje najgorszy akceptowalny czas odpowiedzi.
3. **Czas do skalowania** (t_{scale}) — czas od momentu przekroczenia progu HPA do osiągnięcia docelowej liczby replik. Definiowany jako $t_{\text{target_replicas}} - t_{\text{threshold_crossed}}$.
4. **Efektywność skalowania** (E) — stosunek względnej zmiany przepustowości do względnej zmiany liczby replik:

$$E = \frac{\Delta T / T_{\min}}{\Delta R / R_{\min}} \quad (5.1)$$

gdzie $T_{\min}$ i $T_{\max}$ to minimalna i maksymalna przepustowość w trakcie testu, a $R_{\min}$ i $R_{\max}$ to minimalna i maksymalna liczba replik. Wartość $E = 1$ oznacza skalowanie liniowe; $E < 1$ oznacza malejące korzyści z dodawania replik; $E > 1$ oznacza ponadliniową poprawę (rzadkie).

5. **Stopa błędów** (e) — odsetek żądań zakończonych kodem HTTP ≥ 400 :

$$e = \frac{\text{failed_requests}}{\text{total_requests}} \times 100\% \quad (5.2)$$

6. **Amplituda skalowania** (A_R) — różnica między maksymalną a minimalną liczbą replik w trakcie testu: $A_R = R_{\max} - R_{\min}$. Większa amplituda oznacza, że HPA zareagował na obciążenie.
7. **Średnie zużycie CPU** — średnie zużycie procesora w milijednostkach (mCPU) na Pod, uśrednione po wszystkich Podach w trakcie testu.

5.2 Scenariusze testowe

5.2.1 Scenariusze bazowe (bez HPA)

Scenariusze `baseline-cpu` i `baseline-io` testują zachowanie systemu bez mechanizmu autoskalowania — liczba replik jest stała (1). Służą jako punkt odniesienia do oceny, czy HPA w ogóle poprawia wydajność.

Parametr	baseline-cpu	baseline-io
Serwis	cpu-service	io-service
HPA	brak	brak
Repliki (stałe)	1	1
Skrypt k6	cpu-load-test.js	io-load-test.js
Profil obciążenia	Fibonacci 1→50 VU + image processing 10 VU	stałe zapytania 1→100 VU + stres-test + external

Tabela 5.1 Parametry scenariuszy bazowych

5.2.2 Scenariusze CPU-bound

Parametr	cpu-cpu	cpu-memory	cpu-custom
Serwis	cpu-service	cpu-service	cpu-service
HPA	CPU 50%	Memory 70%	Custom RPS 100/Pod
minReplicas	2	2	2
maxReplicas	10	10	10
Skrypt k6	cpu-load-test.js	cpu-load-test.js	cpu-load-test.js

Tabela 5.2 Parametry scenariuszy CPU-bound

5.2.3 Scenariusze I/O-bound

Parametr	io-cpu	io-memory	io-custom
Serwis	io-service	io-service	io-service
HPA	CPU 50%	Memory 70%	Custom RPS 100/Pod
minReplicas	2	2	2
maxReplicas	10	10	10
Skrypt k6	io-load-test.js	io-load-test.js	io-load-test.js

Tabela 5.3 Parametry scenariuszy I/O-bound

5.3 Wyniki eksperymentalne — środowisko GKE

5.3.1 Scenariusze bazowe

Metryka	baseline-cpu	baseline-io
Średni RPS	TODO	TODO
p50 (ms)	TODO	TODO
p95 (ms)	TODO	TODO
p99 (ms)	TODO	TODO
Stopa błędów (%)	TODO	TODO
Średnie zużycie CPU (mCPU)	TODO	TODO
Średnie zużycie pamięci (MiB)	TODO	TODO

Tabela 5.4 Wyniki scenariuszy bazowych (średnia z 5 powtórzeń)

5.3.2 Scenariusze CPU-bound — porównanie strategii HPA

Metryka	cpu-cpu	cpu-memory	cpu-custom
Średni RPS	TODO	TODO	TODO
p50 (ms)	TODO	TODO	TODO
p95 (ms)	TODO	TODO	TODO
p99 (ms)	TODO	TODO	TODO
Stopa błędów (%)	TODO	TODO	TODO
Amplituda skalowania (A_R)	TODO	TODO	TODO
Czas do skalowania (s)	TODO	TODO	TODO
Efektywność skalowania (E)	TODO	TODO	TODO
Maks. liczba replik	TODO	TODO	TODO
Średnie zużycie CPU (mCPU)	TODO	TODO	TODO

Tabela 5.5 Wyniki scenariuszy CPU-bound (średnia z 5 powtórzeń)

5.3.3 Scenariusze I/O-bound — porównanie strategii HPA

Metryka	io-cpu	io-memory	io-custom
Średni RPS	TODO	TODO	TODO
p50 (ms)	TODO	TODO	TODO
p95 (ms)	TODO	TODO	TODO
p99 (ms)	TODO	TODO	TODO
Stopa błędów (%)	TODO	TODO	TODO
Amplituda skalowania (A_R)	TODO	TODO	TODO
Czas do skalowania (s)	TODO	TODO	TODO
Efektywność skalowania (E)	TODO	TODO	TODO
Maks. liczba replik	TODO	TODO	TODO
Średnie zużycie CPU (mCPU)	TODO	TODO	TODO

Tabela 5.6 Wyniki scenariuszy I/O-bound (średnia z 5 powtórzeń)

5.3.4 Wykresy przebiegu skalowania

Poniższe wykresy będą generowane z danych CSV eksportowanych z *Grafany*:

Rysunek 5.1 Przebieg skalowania dla scenariusza `cpu-cpu` — liczba replik HPA w czasie (źródło: `replicas.csv`)

Rysunek 5.2 Przebieg skalowania dla scenariusza `io-custom` — liczba replik HPA w czasie (źródło: `replicas.csv`)

5.3.5 Analiza zjawisk anomalnych

W trakcie eksperymentów na GKE szczególna uwaga zostanie poświęcona trzem zjawiskom przejściowym, które mają istotny wpływ na zachowanie systemów *SaaS* w śro-

dowisku chmurowym. Każde z tych zjawisk jest identyfikowane na podstawie danych z eksportów CSV z *Grafany* oraz logów *k6*.

HPA Lag — opóźnienie reakcji autoskalera

HPA Lag to opóźnienie między przekroczeniem progu metryki a faktycznym utworzeniem i osiągnięciem gotowości przez nowe Pody. Opóźnienie to wynosi ~60–90 sekund i składa się z następujących faz:

1. **Scrapowanie metryk** (~0–15 s) — *Prometheus* zbiera metryki w interwale 15-sekundowym. Nowa wartość metryki pojawi się w *Prometheusie* najpóźniej po jednym pełnym cyklu scrapowania.
2. **Odświeżenie HPA** (~0–15 s) — kontroler HPA pobiera metryki co 15 sekund. Decyzja o skalowaniu może być opóźniona o ten interwał.
3. **Tworzenie Podów** (~2–5 s) — *kube-scheduler* przydziela Poda do węzła, *kubelet* rozpoczyna tworzenie kontenera.
4. **Cold Start** (~15–30 s) — pobranie obrazu z *GCP Artifact Registry*, uruchomienie kontenera, zaliczenie *readiness probe*. W tym oknie Pod jest już widoczny dla *LoadBalancer*a, ale nie obsługuje jeszcze ruchu.

W okresie *HPA Lag* istniejące Pody są przeciążone (przekroczyły próg metryki), a nowe repliki nie są jeszcze gotowe. Prowadzi to do podwyższonych opóźnień percentyla P99 oraz potencjalnych błędów HTTP 502/503. Zjawisko to jest dokumentowane dla każdego scenariusza na podstawie przesunięcia czasowego między krzywą RPS (rosnącą) a krzywą liczby replik (opóźnioną) w danych CSV.

Cold Start Latency — opóźnienie zimnego startu Podów

Cold Start Latency to czas potrzebny nowo utworzonemu Podowi na osiągnięcie gotowości do obsługi ruchu. Obejmuje:

- **Pobranie obrazu** — ściągnięcie obrazu kontenera z *GCP Artifact Registry* do lokalnego cache *containerd* na węźle. W przypadku pierwszego uruchomienia na danym węźle (lub nowej wersji obrazu z *imagePullPolicy: Always*) czas ten wynosi 5–20 sekund w zależności od rozmiaru obrazu i przepustowości sieci.
- **Start kontenera** — uruchomienie procesu *Uvicorn*, inicjalizacja aplikacji *FastAPI*, załadowanie zależności — ~2–5 sekund.
- **Readiness probe** — pierwsza sonda *readiness* (GET /health) wykonywana jest z opóźnieniem *initialDelaySeconds=5* i okresem *periodSeconds=10*. Pod jest uznawany za gotowy po pierwszym sukcesie sondy.

Łączny czas *Cold Start Latency* wynosi 15–30 sekund. W tym oknie *GCP LoadBalancer* może już kierować ruch do niedojrzałego Poda (przed zaliczeniem *readiness probe*), co skutkuje błędami HTTP 502/503 rejestrowanymi przez *k6*. Zjawisko to jest szczególnie widoczne w pierwszych minutach po decyzji HPA o skalowaniu w górę.

CPU Throttling Cascade — kaskadowe błędy przy throttlingu CPU

CPU Throttling Cascade to zjawisko kaskadowej degradacji wydajności, gdy Pody osiągają limit CPU podczas gwałtownego wzrostu ruchu. Mechanizm przebiega następująco:

1. **Wzrost ruchu** — liczba żądań rośnie (scenariusz ramp-up w *k6*).
2. **Przekroczenie CPU limit** — Pody osiągają limit CPU (`cpu-service: 1000m`, `io-service: 500m`). Mechanizm *CFS bandwidth control* w jądrze Linux ogranicza czas procesora dla kontenera (*CPU throttling*).
3. **Kolejkowanie żądań** — ograniczony czas CPU powoduje, że *Uvicorn* nie nadąża z przetwarzaniem żądań. Żądania są kolejkowane, co zwiększa czas odpowiedzi.
4. **Timeouty** — czas odpowiedzi przekracza timeout *k6* (domyślnie 60s) lub timeout *LoadBalancer*. *k6* rejestruje błędy typu `dial: i/o timeout` lub `request timeout`.
5. **Efekt kaskadowy** — kolejkowane żądania blokują wątki robocze *Uvicorn*, uniemożliwiając obsługę nowych żądań. Powstaje dodatnia pętla sprzężenia zwrotnego: im więcej żądań jest kolejkowanych, tym mniejsza jest zdolność do ich obsługi.
6. **Ustąpienie** — degradacja trwa do momentu osiągnięcia gotowości przez nowe Pody (utworzone przez HPA). Gdy nowe repliki zaczynają obsługiwać ruch, obciążenie na Pod spada poniżej limitu CPU i wydajność wraca do normy.

CPU Throttling Cascade jest szczególnie niebezpieczny w systemach *SaaS*, gdzie jeden przeciążony tenant może spowodować kaskadową degradację dla wszystkich tenantów współdzielących infrastrukturę. Zjawisko to jest analizowane na podstawie korelacji między danymi z `cpu_mcpu.csv` (zużycie CPU zbliżające się do limitu) a `latency_ms.csv` (gwałtowny wzrost opóźnień) oraz metrykami błędów z *k6*.

5.4 Analiza wyników

5.4.1 Porównanie strategii HPA dla CPU-bound

Oczekiwany rezultat: Dla obciążenia *CPU-bound*, strategia HPA CPU powinna być najskuteczniejsza, ponieważ zużycie CPU silnie koreluje z liczbą żądań. Strategia HPA Memory powinna być nieskuteczna (brak skalowania), ponieważ pamięć jest alokowana statycznie i nie zmienia się pod obciążeniem. Strategia HPA Custom RPS powinna być skuteczna, ponieważ RPS rośnie proporcjonalnie do obciążenia.

5.4.2 Porównanie strategii HPA dla I/O-bound

Oczekiwany rezultat: Dla obciążenia *I/O-bound*, strategia HPA CPU powinna być nieskuteczna (brak skalowania), ponieważ zużycie CPU nie koreluje z liczbą żądań — serwis spędza czas na `asyncio.sleep()`. Strategia HPA Memory również powinna być nieskuteczna. Strategia HPA Custom RPS powinna być najskuteczniejsza, ponieważ RPS rośnie proporcjonalnie do obciążenia niezależnie od profilu aplikacji.

5.4.3 Efektywność skalowania

Scenariusz	Efektywność E	Amplituda A_R	Czas skalowania (s)
cpu-cpu	TODO	TODO	TODO
cpu-memory	TODO	TODO	TODO
cpu-custom	TODO	TODO	TODO
io-cpu	TODO	TODO	TODO
io-memory	TODO	TODO	TODO
io-custom	TODO	TODO	TODO

Tabela 5.7 Efektywność skalowania — porównanie strategii

5.4.4 Analiza zjawisk anomalnych — podsumowanie obserwacji

Na podstawie danych z 5 eksportów CSV na scenariusz możliwe będzie ilościowe scharakteryzowanie trzech zjawisk anomalnych:

- **HPA Lag** — mierzone jako przesunięcie czasowe między początkiem wzrostu RPS (`rps.csv`) a początkiem wzrostu liczby replik (`replicas.csv`). Oczekiwana wartość: $\sim 60\text{--}90\text{ s}$, zależna od interwałów scrapowania *Prometheusa* i odświeżania HPA.
- **Cold Start Latency** — mierzone jako czas między pierwszym pojawieniem się nowej repliki w `replicas.csv` a spadkiem opóźnień P99 (`latency_ms.csv`) do poziomu sprzed skalowania. Oczekiwana wartość: $\sim 15\text{--}30\text{ s}$.
- **CPU Throttling Cascade** — identyfikowane przez jednoczesne wystąpienie trzech warunków: (1) zużycie CPU per Pod zbliża się do limitu (`cpu_mcpu.csv` $\approx 1000\text{ m}$ dla `cpu-service`), (2) gwałtowny wzrost opóźnień P99 (`latency_ms.csv`), (3) wzrost stopy błędów w metrykach *k6*. Oczekiwany czas trwania: do momentu osiągnięcia gotowości przez nowe Pody.

5.5 Dyskusja i interpretacja

Niniejsza sekcja będzie zawierać:

- Analizę porównawczą skuteczności strategii HPA w środowisku GKE.
- Ilościową charakterystykę zjawisk anomalnych (*HPA Lag*, *Cold Start Latency*, *CPU Throttling Cascade*) na podstawie danych z eksportów CSV z *Grafany*.
- Walidację lub falsyfikację hipotezy badawczej.
- Dyskusję implikacji wyników dla systemów *SaaS* — szczególnie wpływu zjawisk anomalnych na współdzieloną infrastrukturę.
- Ograniczenia metodologii (np. sztuczny charakter obciążenia, pojedyncza strefa GCP, stałe progi HPA).

5.6 Wnioski z badań

Oczekiwane wnioski:

1. **HPA CPU** jest skuteczne wyłącznie dla aplikacji *CPU-bound*, gdzie zużycie procesora koreluje z obciążeniem. Dla aplikacji *I/O-bound* HPA CPU nie reaguje na wzrost ruchu.
2. **HPA Memory** jest nieskuteczne dla obu profili obciążeniowych — pamięć nie zmienia się proporcjonalnie do liczby żądań w testowanych aplikacjach.
3. **HPA Custom RPS** jest uniwersalnie skuteczne — niezależnie od profilu aplikacji, liczba żądań na sekundę koreluje z obciążeniem, co czyni tę strategię najbardziej odporną na różnice w charakterystyce aplikacji.
4. **Wybór metryki HPA musi być dopasowany do profilu aplikacji** — hipoteza badawcza zostaje potwierdzona.
5. **Zjawiska anomalne** (*HPA Lag*, *Cold Start Latency*, *CPU Throttling Cascade*) mają istotny wpływ na wydajność systemów *SaaS* w GKE — szczególnie w okresach gwałtownego wzrostu obciążenia. Świadomość tych zjawisk i ich ilościowa charakterystyka są niezbędne do prawidłowego projektowania polityk autoskalowania i progów akceptacji SLA.

Rozdział 6

Rozszerzenie *SaaS Multi-Tenancy*

6.1 Status implementacji

Zgodnie z planem pracy (Faza VIII), rozszerzenie systemu o model *SaaS Multi-Tenancy* zostało zaplanowane jako końcowy etap projektu. Na obecnym etapie prac (stan na czerwiec 2026) funkcjonalność *multi-tenancy* **nie została zaimplementowana** w kodzie źródłowym. W repozytorium nie występują nagłówki `X-Tenant-ID`, mechanizmy izolacji danych między tenantami ani konfiguracje *namespace-per-tenant*.

Niniejszy rozdział przedstawia zaprojektowaną koncepcję rozszerzenia *SaaS*, która stanowi podstawę do przyszłej implementacji. Koncepcja uwzględnia specyfikę środowiska GKE — zarządzanego klastra z natywną integracją z GCP — i zakłada wykorzystanie mechanizmów izolacji dostępnych w *Kubernetes* (*ResourceQuota*, *NetworkPolicy*, *Namespace*) zamiast rozwiązań lokalnych.

6.2 Koncepcja *Multi-Tenancy* w środowisku GKE

6.2.1 Model izolacji tenantów

Zaprojektowano dwa poziomy izolacji tenantów — lekki (*application-level*) i pełny (*namespace-per-tenant*):

1. **Application-level multi-tenancy** — tenanty są rozróżniani na poziomie aplikacji przez nagłówek `X-Tenant-ID`. Wszyscy tenanty współdzielą ten sam *Deployment* i *namespace*. Izolacja dotyczy wyłącznie danych (np. klucze cache w *memory-service* są prefiksowane identyfikatorem tenanta), natomiast zasoby obliczeniowe są współdzielone. Ten poziom jest najprostszy w implementacji i najbardziej ekonomiczny — nie wymaga dodatkowych zasobów *Kubernetes* per tenant. Nie zapewnia jednak izolacji wydajnościowej: „głośny sąsiad” (*noisy neighbor*) może zdegradować wydajność innych tenantów.
2. **Namespace-per-tenant** — każdy tenant otrzymuje dedykowany *namespace* (np. `tenant-001`, `tenant-002`) z własnymi *Deploymentami*, *HPA*, *ResourceQuota* i *NetworkPolicy*. Zapewnia to pełną izolację zasobów i wydajności. W środowisku GKE model ten jest skalowalny dzięki automatycznemu skalowaniu węzłów (*cluster autoscaler*) — w przeciwieństwie do lokalnego klastra na pojedynczym serwerze, gdzie zasoby są ograniczone fizycznym sprzętem.

6.2.2 Projektowana architektura

W modelu *application-level multi-tenancy*:

- **Nagłówek X-Tenant-ID** — każdy mikroserwis odczytuje identyfikator tenanta z nagłówka HTTP. W przypadku jego braku, żądanie jest odrzucane z kodem 400.
- **Izolacja danych** — *memory-service* używa prefiksów kluczy (`<tenant_id>:<key>`), zapewniając logiczną separację danych w obrębie jednego *in-memory cache*.
- **Metryki per tenant** — *Prometheus* zbiera metryki z etykietą `tenant_id`, umożliwiając analizę wydajności w podziale na tenantów.
- **HPA z etykietami** — *HorizontalPodAutoscaler* uwzględnia obciążenie od wszystkich tenantów łącznie, co odzwierciedla realistyczny scenariusz *SaaS*, gdzie infrastruktura jest współdzielona.

W modelu *namespace-per-tenant*:

- Każdy tenant ma własny *namespace* (np. `tenant-001`, `tenant-002`) z kompletnym zestawem zasobów.
- *ResourceQuota* ogranicza sumaryczne zużycie CPU i pamięci per tenant:

```

1 apiVersion: v1
2 kind: ResourceQuota
3 metadata:
4   name: tenant-quota
5   namespace: tenant-001
6 spec:
7   hard:
8     requests.cpu: "2000m"
9     requests.memory: "2Gi"
10    limits.cpu: "4000m"
11    limits.memory: "4Gi"
12    pods: "15"

```

Listing 6.1 Przykładowa *ResourceQuota* per tenant

- *NetworkPolicy* izoluje ruch sieciowy między tenantami — Pody w namespace `tenant-001` nie mogą komunikować się z Podami w `tenant-002`, z wyjątkiem jawnie dozwolonych ścieżek.
- Każdy tenant ma niezależny HPA, co umożliwia precyzyjne skalowanie per tenant — przeciążenie jednego tenanta nie wpływa na decyzje HPA dla innych tenantów.

6.3 Rozważania dotyczące wdrożenia na GKE

Wdrożenie modelu *namespace-per-tenant* na GKE różni się istotnie od wdrożenia na lokalnym klastrze:

- **Skalowalność węzłów** — *GKE cluster autoscaler* automatycznie dodaje węzły, gdy liczba Podów przekracza dostępne zasoby. Eliminuje to ograniczenie sprzętowe występujące na pojedynczym serwerze fizycznym (np. QNAP z 8 GB RAM), gdzie wszystkie Pody muszą zmieścić się w stałej puli zasobów.

- **Koszt infrastruktury** — w modelu *pay-as-you-go* GCP, każdy dodatkowy węzeł generuje koszt. Izolacja *namespace-per-tenant* jest zatem droższa niż *application-level*, ale zapewnia lepszą izolację. Analiza kosztowa różnych strategii HPA w kontekście *multi-tenancy* stanowi potencjalny kierunek dalszych badań.
- **LoadBalancer per tenant** — w modelu *namespace-per-tenant* każdy tenant może wymagać osobnego *LoadBalancer*a, co zwiększa koszty i złożoność. Alternatywnie, pojedynczy *Ingress* (*GKE Ingress Controller*) może rutować ruch na podstawie nagłówka *Host* lub ścieżki, eliminując potrzebę wielu *LoadBalancer*ów.
- **Eksplzja metryk** — metryki z etykietą *tenant_id* zwiększają kardynalność szeregów czasowych w *Prometheusie*. Przy dużej liczbie tenantów może to prowadzić do przeciążenia *Prometheusa*. Rozwiązaniem jest agregacja metryk na poziomie *recording rules* lub użycie *Thanos/Cortex* do długoterminowego przechowywania metryk.

6.4 Planowane scenariusze testowe

Po zaimplementowaniu rozszerzenia *multi-tenancy* planowane jest przeprowadzenie dodatkowych testów na klastrze GKE:

1. **Test izolacji wydajnościowej** — symulacja scenariusza „głośnego sąsiada” (*noisy neighbor*), w którym jeden tenant generuje wysokie obciążenie (ramp-up *k6* do 50 VU), a drugi wysyła stały, niski ruch (5 VU). Celem jest sprawdzenie, czy HPA chroni „cichego” tenanta przed degradacją wydajności. Test przeprowadzony zostanie dla obu modeli izolacji (*application-level* i *namespace-per-tenant*).
2. **Test skalowania z tenantami** — porównanie efektywności HPA w wariancie z jednym tenantem vs. z wieloma tenantami (2, 5, 10). Oczekuje się, że HPA Custom RPS zachowa skuteczność niezależnie od liczby tenantów, podczas gdy HPA CPU może wykazywać degradację przy wielu tenantach o różnych profilach.
3. **Test kosztu izolacji** — porównanie zużycia zasobów (liczba węzłów GKE, łączne CPU i pamięć) w modelu *application-level* vs. *namespace-per-tenant* przy tej samej liczbie tenantów i poziomie obciążenia.
4. **Test odporności na zjawiska anomalne** — weryfikacja, czy zjawiska *HPA Lag*, *Cold Start Latency* i *CPU Throttling Cascade* (opisane w Rozdziale 5) nasilają się w środowisku *multi-tenant*. W szczególności: czy *noisy neighbor* może wywołać *CPU Throttling Cascade* u innych tenantów współdzielących ten sam *Deployment*.

6.5 Oczekiwane wyzwania

- **Narzut nagłówka X-Tenant-ID** — każdy mikroservis musi walidować i propagować nagłówek, co wymaga modyfikacji kodu wszystkich serwisów. W modelu *application-level* nagłówek musi być przekazywany między serwisami (np. z *io-service* do *echo-server*), co wymaga instrumentacji klienta HTTP (*httpx.AsyncClient*).
- **Eksplzja metryk** — metryki z etykietą *tenant_id* zwiększają kardynalność szeregów czasowych, co może przeciążyć *Prometheusa* przy dużej liczbie tenantów. Rozwiązaniem jest ostrożne zarządzanie etykietami i ewentualne użycie *recording rules* do preagregacji.

- **Synchronizacja HPA w modelu application-level** — przy wielu tenantach współdzielących *Deployment*, HPA może „przeskalować” w odpowiedzi na jednego „głośnego” tenanta, niepotrzebnie zwiększając koszty dla wszystkich. Jest to inherentna wada modelu *application-level*, która motywuje model *namespace-per-tenant* dla zastosowań produkcyjnych.
- **Złożoność konfiguracji namespace-per-tenant** — każdy nowy tenant wymaga utworzenia *namespace*, *Deploymentów*, *Serwisów*, *HPA*, *ResourceQuota* i *Network-Policy*. Automatyzacja tego procesu (np. przez *Helm chart* per tenant lub operator *Kubernetes*) jest niezbędna dla skalowalności operacyjnej.

6.6 Podsumowanie

Rozszerzenie *SaaS Multi-Tenancy* stanowi logiczną kontynuację badań nad skalowalnością — po ustaleniu, która strategia HPA jest najskuteczniejsza dla danego profilu obciążeniowego, kolejnym krokiem jest sprawdzenie, czy wnioski te utrzymują się w środowisku wielodostępnym (*multi-tenant*). Środowisko GKE, z automatycznym skalowaniem węzłów i natywną integracją z ekosystemem GCP, jest naturalną platformą do tych badań — eliminuje bowiem ograniczenia sprzętowe występujące na pojedynczym serwerze lokalnym i umożliwia testowanie z realistyczną liczbą tenantów.

Implementacja tego rozszerzenia jest zaplanowana jako Faza VIII projektu i nie została jeszcze zrealizowana.

Rozdział 7

Podsumowanie

7.1 Podsumowanie zrealizowanych prac

W ramach niniejszej pracy magisterskiej zrealizowano następujące cele:

1. **Analiza literatury** — przeprowadzono przegląd istniejących badań nad autoskalowaniem w *Kubernetes*, identyfikując pięć kluczowych luk badawczych: brak systematycznego porównania strategii HPA dla różnych profili obciążeniowych, ograniczoną analizę HPA Memory, brak badań łączących HPA z modelem *SaaS*, brak analizy zjawisk anomalnych w autoskalowaniu chmurowym oraz brak otwartego, odtwarzalnego środowiska badawczego.
2. **Zaprojektowanie architektury** — zaprojektowano system badawczy oparty na macierzy 2×3 (dwa profile obciążeniowe $\times$ trzy strategie HPA), składający się z czterech mikroserwisów (*cpu-service*, *io-service*, *echo-server*, *memory-service*), infrastruktury monitorującej opartej na *kube-prometheus-stack* (Helm) oraz dedykowanej maszyny *Compute Engine* jako generatora obciążenia.
3. **Implementacja** — zaimplementowano kompletny system badawczy w języku Python 3.12 z frameworkiem *FastAPI*, konteneryzowany za pomocą *Docker* i wdrażany na zarządzanej klastrze *Google Kubernetes Engine* (GKE). Wszystkie manifesty *Kubernetes* (Deploymenty, Serwisy typu *LoadBalancer*, HPA) oraz konfiguracje monitoringu są dostępne w repozytorium. Serwisy są eksponowane przez *GCP LoadBalancer*, co odzwierciedla warunki produkcyjne.
4. **Automatyzacja testów** — opracowano skrypt *run-tests.sh* orkiestrujący wykonanie 8 scenariuszy testowych z 5-krotnym powtórzeniem każdego, z automatycznym przełączaniem HPA, resetowaniem *Deploymentów* i cooldownem między scenariuszami. Skrypt działa na maszynie *k6-generator* (*Compute Engine*) w tej samej sieci VPC co klaster GKE.
5. **Scenariusze obciążeniowe** — zdefiniowano trzy skrypty *k6* (*cpu-load-test.js*, *io-load-test.js*, *combined-scenario.js*) z precyzyjnymi progami akceptacji, profilami obciążenia dopasowanymi do charakterystyki aplikacji oraz mapowaniem hostów (*options.hosts*) na adresy IP *GCP LoadBalancer*, eliminującym potrzebę konfiguracji DNS.
6. **Infrastruktura monitorująca** — wdrożono *kube-prometheus-stack* przez *Helm*, zapewniając automatycznie skonfigurowany stos monitorujący (*Prometheus*, *Grafana*, *kube-state-metrics*, *node-exporter*) z *Grafaną* eksponowaną publicznie przez

LoadBalancer. Zdefiniowano procedurę eksportu 5 zestawów danych CSV na scenariusz (`replicas`, `cpu_mcpu`, `memory_mb`, `rps`, `latency_ms`), które wraz z wynikami *k6* tworzą kompletny zbiór danych badawczych.

7. **Charakterystyka zjawisk anomalnych** — zidentyfikowano i opisano trzy kluczowe zjawiska przejściowe wpływające na autoskalowanie w środowisku chmurowym: *HPA Lag* (opóźnienie reakcji autoskalera, ~60–90 s), *Cold Start Latency* (opóźnienie zimnego startu Podów, ~15–30 s) oraz *CPU Throttling Cascade* (kaskadowe błędy przy throttlingu CPU). Dla każdego zjawiska zaproponowano metodę ilościowej analizy na podstawie danych z eksportów CSV z *Grafany*.

7.2 Weryfikacja hipotezy badawczej

Hipoteza badawcza postawiona w Rozdziale 1 brzmiała:

„Nie istnieje jedna uniwersalna metryka HPA — skuteczność strategii skalowania zależy od charakterystyki obciążeniowej aplikacji (workload profile). Błędny wybór metryki prowadzi do niedoskalowania lub przeskalowania.”

Na obecnym etapie prac (czerwiec 2026) badania eksperymentalne na klastrze GKE nie zostały jeszcze przeprowadzone, w związku z czym hipoteza nie może zostać formalnie zweryfikowana. Poniżej przedstawiono oczekiwane rezultaty oparte na analizie teoretycznej i charakterystyce zaimplementowanych aplikacji:

- **HPA CPU** — oczekuje się, że będzie skuteczne wyłącznie dla `cpu-service`, gdzie zużycie procesora silnie koreluje z liczbą żądań (operacje *Pillow*, rekurencyjny Fibonacciego). Dla `io-service`, gdzie procesor jest bezczynny podczas `asyncio.sleep()`, HPA CPU nie powinno reagować na wzrost ruchu.
- **HPA Memory** — oczekuje się, że będzie nieskuteczne dla obu serwisów, ponieważ ani `cpu-service`, ani `io-service` nie alokują pamięci dynamicznie w odpowiedzi na obciążenie. Pamięć jest alokowana statycznie przy starcie i utrzymuje stały poziom.
- **HPA Custom RPS** — oczekuje się, że będzie uniwersalnie skuteczne, ponieważ liczba żądań na sekundę rośnie proporcjonalnie do obciążenia niezależnie od tego, czy żądania są CPU-intensywne, czy I/O-intensywne.

Dodatkowo, oczekuje się, że analiza zjawisk anomalnych (*HPA Lag*, *Cold Start Latency*, *CPU Throttling Cascade*) dostarczy ilościowych danych na temat wpływu tych zjawisk na wydajność systemów *SaaS* w środowisku GKE.

7.3 Ograniczenia pracy

Należy wskazać następujące ograniczenia niniejszej pracy:

- **Brak wyników eksperymentalnych** — na obecnym etapie zaimplementowano kompletny system badawczy i narzędzia do testów, ale same badania nie zostały jeszcze przeprowadzone. Wnioski przedstawione w Rozdziale 5 mają charakter oczekiwany i wymagają walidacji empirycznej.

- **Sztuczny charakter obciążenia** — aplikacje badawcze (*cpu-service*, *io-service*) symulują określone profile obciążeniowe, ale nie odzwierciedlają rzeczywistych aplikacji produkcyjnych. Rzeczywiste systemy często wykazują mieszane profile (*CPU-bound* + *I/O-bound*).
- **Ograniczony zakres HPA** — badanie ogranicza się do trzech metryk HPA (*CPU*, *Memory*, *Custom RPS*). Nie analizowano innych strategii, takich jak HPA na podstawie liczby żądań w toku (*in-flight*), metryk zewnętrznych (*External Metrics*) ani skalowania predykcyjnego.
- **Pojedyncza strefa GCP** — wszystkie eksperymenty są przeprowadzane w jednej strefie GCP (*europa-central2*), co oznacza, że nie badano wpływu opóźnień między strefami (*cross-zone latency*) ani scenariuszy *multi-region*.
- **Brak implementacji *multi-tenancy*** — rozszerzenie *SaaS Multi-Tenancy* (Faza VIII) zostało zaprojektowane koncepcyjnie, ale nie zaimplementowane w kodzie. Wnioski dotyczące zachowania systemu w środowisku wielodostępnym pozostają hipotetyczne.
- **Stałe progi HPA** — progi HPA (CPU 50%, Memory 70%, RPS 100/Pod) zostały ustalone arbitralnie na podstawie wartości referencyjnych. Nie badano wpływu różnych wartości progowych na skuteczność skalowania.

7.4 Rekomendacje praktyczne

Na podstawie analizy teoretycznej i projektu systemu sformułowano następujące rekomendacje dla praktyków:

1. **Dobieraj metrykę HPA do profilu aplikacji** — przed wyborem strategii autoskalowania należy scharakteryzować aplikację pod kątem profilu obciążeniowego (*CPU-bound*, *I/O-bound*, *memory-bound*). Domyślna strategia HPA CPU może być nieskuteczna dla aplikacji *I/O-bound*.
2. **Używaj HPA Custom RPS jako strategii uniwersalnej** — jeśli charakterystyka aplikacji jest nieznana lub zmienna, metryka RPS (dostarczana przez *prometheus-adapter*) jest bezpiecznym wyborem, ponieważ koreluje z obciążeniem niezależnie od profilu.
3. **Unikaj HPA Memory jako jedynej metryki** — chyba że aplikacja wykazuje silną korelację między obciążeniem a zużyciem pamięci (np. cache, kolejki), HPA Memory prawdopodobnie nie będzie skuteczne.
4. **Uwzględnij zjawiska anomalne w SLA** — *HPA Lag* (~60–90 s) i *Cold Start Latency* (~15–30 s) oznaczają, że system potrzebuje 1–2 minut na reakcję na nagły wzrost obciążenia. Progi SLA powinny być definiowane z uwzględnieniem tego opóźnienia — oczekiwanie, że system zareaguje w kilka sekund na gwałtowny skok ruchu, jest nierealistyczne przy HPA opartym na metrykach z *Prometheusa*.
5. **Monitoruj CPU throttling** — *CPU throttling* jest „cichym zabójcą” wydajności — nie powoduje awarii, ale drastycznie wydłuża czasy odpowiedzi. Rekomenduje się monitorowanie metryki `container_cpu_cfs_throttled_seconds_total` i ustawienie alertów przy jej wzroście.

6. **Używaj GCP LoadBalancer zamiast port-forward** — dla testów obciążeniowych i środowisk produkcyjnych, bezpośrednia ekspozycja serwisów przez *GCP LoadBalancer* eliminuje wąskie gardło tunelowania przez *API server* i zapewnia realistyczne warunki sieciowe.
7. **Wdróż monitoring przez Helm** — *kube-prometheus-stack* przez *Helm* redukuje złożoność konfiguracji monitoringu i zapewnia spójność wersji komponentów, co jest szczególnie istotne w środowiskach produkcyjnych.

7.5 Dalsze kierunki badań

Niniejsza praca otwiera następujące kierunki dalszych badań:

1. **Przeprowadzenie badań eksperymentalnych na GKE** — najpilniejszym kolejnym krokiem jest wykonanie 8 scenariuszy testowych na klastrze GKE i zebranie danych empirycznych do weryfikacji hipotezy. Środowisko badawcze, narzędzia i procedury są w pełni przygotowane.
2. **Ilościowa analiza zjawisk anomalnych** — wykorzystanie 5 eksportów CSV na scenariusz do ilościowego scharakteryzowania *HPA Lag*, *Cold Start Latency* i *CPU Throttling Cascade*. Dane te mogą posłużyć do opracowania modeli predykcyjnych wykrywających te zjawiska.
3. **Implementacja *multi-tenancy*** — zrealizowanie Fazy VIII zgodnie z koncepcją opisaną w Rozdziale 6 i przeprowadzenie testów w środowisku wielodostępnym na GKE.
4. **Rozszerzenie macierzy badawczej** — dodanie *memory-service* jako trzeciego profilu obciążeniowego (*memory-bound*) i zbadanie, czy HPA Memory staje się skuteczne dla aplikacji, których zużycie pamięci koreluje z obciążeniem.
5. **Badanie innych metryk HPA** — analiza skuteczności metryk takich jak liczba żądań w toku (*http_requests_inflight*), metryk zewnętrznych (długość kolejki *RabbitMQ*, liczba wiadomości *Kafka*) oraz kombinacji wielu metryk w jednym HPA.
6. **Skalowanie predykcyjne** — porównanie reaktywnego HPA z podejściami predykcyjnymi (np. opartymi na LSTM), które przewidują przyszłe obciążenie i skalują proaktywnie, potencjalnie eliminując problem *HPA Lag*.
7. **Analiza kosztowa** — rozszerzenie badań o analizę kosztów infrastruktury GCP w modelu *pay-as-you-go* dla różnych strategii HPA, szczególnie w kontekście *multi-tenancy*, gdzie przeskalowanie przekłada się bezpośrednio na wyższe koszty.
8. **Testy wielostrefowe** — powtórzenie eksperymentów w konfiguracji *multi-zone* GKE w celu zbadania wpływu opóźnień między strefami na skuteczność HPA i zjawiska anomalne.
9. **Testy z rzeczywistymi aplikacjami** — zastąpienie syntetycznych serwisów badawczych rzeczywistymi aplikacjami (np. system rezerwacji, przetwarzanie obrazu w pipeline ML) w celu walidacji uniwersalności wniosków.

7.6 Podsumowanie końcowe

Niniejsza praca magisterska stanowi kompleksowe przygotowanie do badań empirycznych nad skutecznością strategii *HorizontalPodAutoscaler* w *Kubernetes* dla różnych profili obciążeniowych, w całości w środowisku *Google Cloud Platform*. Zaimplementowano w pełni funkcjonalny system badawczy składający się z mikroservisów, infrastruktury monitorującej opartej na *kube-prometheus-stack* (Helm) oraz zestawu skryptów automatyzujących, gotowy do przeprowadzenia eksperymentów na klastrze GKE.

Główny wkład pracy obejmuje:

- Systematyczną analizę literatury identyfikującą luki badawcze w dziedzinie autoskalowania w *Kubernetes*, w tym nowo zidentyfikowaną lukę dotyczącą analizy zjawisk anomalnych (*HPA Lag*, *Cold Start Latency*, *CPU Throttling Cascade*).
- Projekt macierzy badawczej 2×3 umożliwiającej izolowane porównanie strategii HPA.
- Implementację czterech mikroservisów o kontrolowanych profilach obciążeniowych (*CPU-bound*, *I/O-bound*, *memory-bound*).
- Wdrożenie infrastruktury monitorującej przez *kube-prometheus-stack* (Helm) z nie-standardowymi metrykami przez *prometheus-adapter*.
- Pełną automatyzację procesu badawczego — od wdrożenia po wykonanie testów i eksport danych CSV z *Grafany*.
- Koncepcję rozszerzenia *SaaS Multi-Tenancy* dla środowiska GKE, uwzględniającą specyfikę zarządzanego klastra i mechanizmy izolacji *Kubernetes*.
- Otwarte, odtwarzalne środowisko badawcze dostępne w repozytorium, możliwe do odtworzenia na dowolnym klastrze GKE.

Przeprowadzenie zaplanowanych badań eksperymentalnych i weryfikacja hipotezy badawczej stanowią bezpośredni kolejny krok, do którego niniejsza praca dostarcza wszystkich niezbędnych narzędzi. Szczególnie wartościowym wkładem będzie ilościowa analiza zjawisk anomalnych na podstawie danych z 5 eksportów CSV na scenariusz — wyniki te mogą mieć istotne znaczenie praktyczne dla projektantów systemów *SaaS* działających na *Kubernetes* w chmurze.

Dodatek A

Spis listingów

Bibliografia

- [1] E. Casalicchio, V. Perciballi. Auto-scaling of containers: The impact of relative and absolute metrics. *2021 IEEE 5th International Workshop on Smart Container Technologies and Orchestration*, strony 1–6. IEEE, 2021.
- [2] T.-T. Nguyen, Y.-J. Kim. A proactive autoscaling approach using lstm-based traffic prediction for kubernetes. *IEEE Access*, 8:149486–149500, 2020.

Spis rysunków

3.1	Topologia systemu badawczego w GCP — wszystkie komponenty w jednej sieci VPC	22
5.1	Przebieg skalowania dla scenariusza <code>cpu-cpu</code> — liczba replik HPA w czasie (źródło: <code>replicas.csv</code>)	51
5.2	Przebieg skalowania dla scenariusza <code>io-custom</code> — liczba replik HPA w czasie (źródło: <code>replicas.csv</code>)	51

Spis tabel

2.1	Podsumowanie stosu technologicznego projektu — środowisko GCP/GKE .	16
3.1	Macierz badawcza 2×3 — identyfikatory scenariuszy testowych	17
3.2	Zasoby przydzielone serwisom (<i>resource requests</i> i <i>limits</i>)	25
5.1	Parametry scenariuszy bazowych	49
5.2	Parametry scenariuszy CPU-bound	50
5.3	Parametry scenariuszy I/O-bound	50
5.4	Wyniki scenariuszy bazowych (średnia z 5 powtórzeń)	50
5.5	Wyniki scenariuszy CPU-bound (średnia z 5 powtórzeń)	51
5.6	Wyniki scenariuszy I/O-bound (średnia z 5 powtórzeń)	51
5.7	Efektywność skalowania — porównanie strategii	54